JAVA SDE-2 INTERVIEW PREPARATION SERIES

LEARNING STEP 4 - VOLUME 4 OF 18

Bit Manipulation in Java

From First Bits to SDE-2 Interview Techniques

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Build bit intuition from zero, master Java masks and shifts, derive high-value interview shortcuts, and apply XOR, subset, trie, and packed-state patterns safely.

BITS | MASKS | SHIFTS | XOR | SUBSETS | INTERVIEW SHORTCUTS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `01B — Number Systems Interview Workbook`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This dedicated volume turns the binary foundation from Volume 01 into a progressive Java interview toolkit. It begins with visible bit patterns and safe operators, derives each shortcut instead of asking readers to memorize it, then develops XOR families, subset state, SDE-2 problem patterns, Java APIs, and production-quality bit fields.

Readiness map

Decision	Evidence
START HERE	The state is a small bounded set of yes/no facts; The problem asks about powers of two, parity, set bits, or fixed-width representation; Duplicate counts allow XOR cancellation; Constraints make subset or submask state feasible; A repeated range-XOR or maximum-XOR query appears.
PREPARE FIRST	Java Foundations Volume 03 through operators, loops, arrays, and methods; Time and Space Complexity Volume 02 through logarithmic growth and output-sensitive analysis; Number Systems Volume 01 binary representation and two's-complement basics.
MOVE ON WHEN	Trace Java int and long bit patterns, shifts, promotion, and sign behavior; Derive and implement safe test, set, clear, toggle, low-bit, count, power-of-two, and range-mask techniques; Select and prove XOR, prefix-XOR, subset, submask, range-AND, and maximum-XOR patterns; Choose among primitive masks, BitSet, EnumSet, boolean arrays, and sets; Explain complexity, contracts, edge cases, and SDE-2 follow-ups without relying on unexplained tricks.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

01B - Number Systems Interview Workbook	**YOU ARE HERE** **04 - Bit Manipulation**	05 - Loops and Index Calculations

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Bits and Java Operators from Zero

Where this chapter fits

This is the first Bit Manipulation chapter. It assumes that you can read a Java loop, but it does not assume that bit expressions feel natural yet. Do not rush to XOR tricks. First learn to see the value, the fixed width, and the operator result.

Use this sequence:

1. write the value in binary;
2. label bit positions from right to left starting at zero;
3. align operands to the same width;
4. apply the operator one column at a time; and
5. interpret the result under the problem's signed or unsigned contract.

The Number Systems volume explains base conversion and two's complement in greater depth. This chapter reviews only the part needed to operate on bits safely in Java.

Learning objectives

After this chapter, you should be able to:

- explain what a bit and a bit position mean;
- convert small nonnegative values between decimal and binary;
- distinguish a mathematical integer from its fixed-width Java representation;
- predict `&`, `|`, `^`, `~`, `<<`, `>>`, and `>>>` results;
- explain why `byte`, `short`, and `char` expressions become `int`;
- print a padded 32-bit or 64-bit view for debugging; and
- identify the sign, shift-distance, and literal-width traps interviewers use.

1.1 A bit is one binary decision

A bit has two possible values: zero or one. In a nonnegative binary number, position `i` contributes `2^i` when that bit is one.

TEXT

```
bit positions:  5 4 3 2 1 0
powers of two: 32 16 8 4 2 1
value:          1 0 1 1 0 1
```

The value `101101` is:

TEXT

```
32 + 0 + 8 + 4 + 0 + 1 = 45
```

The rightmost bit is the least significant bit, or LSB. The leftmost bit inside the chosen width is the most significant bit, or MSB. In a signed Java integer, the MSB also participates in the sign under two's-complement interpretation.

Quick conversion: decimal to binary

For a small nonnegative number, repeatedly select powers of two.

Convert 26:

TEXT

```
largest power <= 26 is 16 -> bit 4 is 1, remainder 10
largest power <= 10 is  8 -> bit 3 is 1, remainder  2
largest power <=  2 is  2 -> bit 1 is 1, remainder  0

26 = 16 + 8 + 2 = 11010
```

For a coding interview, you rarely need to perform long manual conversions. You do need to recognize powers of two, align short patterns, and explain what each selected position represents.

Hexadecimal is a compact bit view

One hexadecimal digit represents four bits.

Hex	Binary	Hex	Binary
`0`	`0000`	`8`	`1000`
`1`	`0001`	`9`	`1001`
`2`	`0010`	`A`	`1010`
`3`	`0011`	`B`	`1011`
`4`	`0100`	`C`	`1100`
`5`	`0101`	`D`	`1101`
`6`	`0110`	`E`	`1110`
`7`	`0111`	`F`	`1111`

Therefore `0x2D` is `0010 1101`, which is decimal 45. Hexadecimal makes long masks readable: `0xFF` means eight low one bits, and `0xFFFFL` means sixteen low one bits in a `long` expression.

1.2 Java gives the representation a fixed width

Bitwise operators work on integral primitive values:

Type	Width	Bitwise expression behavior
`byte`	8 bits	promoted to `int` before the operation
`short`	16 bits	promoted to `int` before the operation
`char`	16 bits, unsigned value range	promoted to `int` before the operation
`int`	32 bits	result is normally a 32-bit `int`
`long`	64 bits	result is 64-bit when an operand is `long`

`boolean` supports logical `&`, `|`, and `^`, but a boolean is not an integer bit field. Floating-point values do not support bitwise arithmetic directly.

The representation and the interpretation are different questions

The 32-bit pattern below can be viewed as a signed Java `int` or as an unsigned magnitude:

```text
11111111 11111111 11111111 11111111
```

- As a signed `int`, it is `-1`.
- As an unsigned 32-bit value, it is `4_294_967_295`.

The bits did not change. Only the interpretation changed. Java's ordinary `int` comparisons are signed. When a problem defines unsigned ordering, use APIs such as `Integer.compareUnsigned`, `Integer.toUnsignedLong`, and `Integer.divideUnsigned` rather than pretending the sign bit is absent.

1.3 Two's complement without mystery

For a fixed width, negate a value by inverting all bits and adding one.

Using an illustrative 8-bit width:

```text
  5 = 0000 0101
 ~5 = 1111 1010
 +1 = 1111 1011 = -5 in 8-bit two's complement
```

Java performs this over 32 bits for `int` and 64 bits for `long`. The most negative value has no positive counterpart in the same signed type:

```text
Integer.MIN_VALUE = 10000000 00000000 00000000 00000000
```

Consequences:

- `-Integer.MIN_VALUE` is still `Integer.MIN_VALUE` because ordinary integer arithmetic wraps.
- `Math.abs(Integer.MIN_VALUE)` is still negative.
- the isolated lowest bit of `Integer.MIN_VALUE` is the sign-bit mask, which is also negative as an `int`.

Treat a mask as a pattern. A mask does not need to be a positive mathematical number to be valid.

1.4 The four column-wise operators

AND: keep a bit only when both inputs contain it

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

```text
12 = 1100
10 = 1010
&    ----
 8 = 1000
```

Think: filter or retain. AND is used to test bits, clear selected positions, and extract fields.

OR: keep a bit when either input contains it

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

```text
12 = 1100
10 = 1010
|    ----
14 = 1110
```

Think: combine or enable. OR is used to set bits and merge independent flags.

XOR: keep a bit when the inputs differ

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

```text
12 = 1100
10 = 1010
^    ----
 6 = 0110
```

Think: difference or toggle. XOR is used for parity, bit flips, cancellation under exact duplicate contracts, and reversible deltas.

NOT: invert every bit in the width

```text
illustrative 8-bit view
 5 = 0000 0101
~5 = 1111 1010 = -6
```

In two's complement, `~x == -x - 1`. The result is not just an inversion of the visible significant bits; Java inverts all 32 or all 64 positions.

1.5 Shift operators

Left shift <<

`value << distance` moves bits left, discards high bits that leave the width, and fills low positions with zeros.

TEXT

```
  6      = 0000 0110
  6 << 1 = 0000 1100 = 12
```

For a nonnegative value whose shifted result fits, shifting left by `k` is equivalent to multiplying by `2^k`. It is not an overflow-safe multiplication technique.

JAVA

```java
int wrapped = 1 << 31;       // Integer.MIN_VALUE, not positive 2^31
long correct = 1L << 31;     // positive 2^31 in a long
```

Arithmetic right shift >>

`>>` preserves the sign interpretation by copying the sign bit into new high positions.

TEXT

```
illustrative 8-bit view
  40      = 0010 1000
  40 >> 2 = 0000 1010 = 10

 -40      = 1101 1000
 -40 >> 2 = 1111 0110 = -10
```

For negative odd values, `x >> 1` rounds toward negative infinity, while Java integer division `x / 2` truncates toward zero.

JAVA

```java
System.out.println(-3 >> 1);  // -2
System.out.println(-3 / 2);   // -1
```

Do not substitute shifts for division without checking the negative-input contract.

Logical right shift >>>

`>>>` fills high positions with zeros, regardless of the sign bit.

```java
System.out.println(-1 >> 1);   // -1
System.out.println(-1 >>> 1);  // 2147483647
```

Use `>>>` when moving through the raw representation, extracting an unsigned field, reversing bits, or scanning all positions without copying the sign.

Java masks the shift distance

An `int` uses only the low 5 bits of the shift distance. A `long` uses only the low 6.

```java
System.out.println(1 << 32);    // 1, because 32 becomes 0
System.out.println(1L << 64);   // 1, because 64 becomes 0
System.out.println(1 << -1);    // same effective distance as 31
```

This is specified Java behavior, not validation. If a parameter represents a bit index, reject values outside `0..31` or `0..63` before shifting.

1.6 Promotion changes small-type expressions

`byte`, `short`, and `char` are promoted to `int` before unary and most binary numeric operations.

```java
byte flags = 0b0000_0101;
int inverted = ~flags;
System.out.println(inverted);   // -6, a 32-bit int result
```

If the intended result is an unsigned 8-bit pattern, mask the promoted result:

```java
int invertedByte = (~flags) & 0xFF;
System.out.println(invertedByte); // 250
```

The mask `0xFF` keeps only the low eight positions.

Literal width determines shift width

```java
long wrong = 1 << 40;    // 32-bit shift; effective distance is 8
long right = 1L << 40;   // 64-bit shift
```

Adding `L` after the computation is too late. The left operand must already be a `long`.

1.7 A reliable way to display bits

`Integer.toBinaryString` omits leading zeros for nonnegative values. Pad it when width matters:

JAVA

```java
static String bits32(int value) {
    return String.format("%32s", Integer.toBinaryString(value))
            .replace(' ', '0');
}

static String bits64(long value) {
    return String.format("%64s", Long.toBinaryString(value))
            .replace(' ', '0');
}
```

Example:

JAVA

```java
System.out.println(bits32(10));
System.out.println(bits32(-1));
```

Expected output:

TEXT

```
00000000000000000000000000001010
11111111111111111111111111111111
```

Use this helper while learning. In an interview, draw only the positions relevant to the invariant unless the sign bit matters.

1.8 Operator reasoning shortcuts that are safe to remember

These are algebraic facts, not problem solutions by themselves:

TEXT

```
x & 0 = 0          x | 0 = x          x ^ 0 = x
x & x = x          x | x = x          x ^ x = 0
x & -1 = x         x | -1 = -1        x ^ -1 = ~x
x & ~x = 0         x | ~x = -1
```

AND, OR, and XOR are associative and commutative. That means grouping and order do not affect a reduction. Only XOR provides pair cancellation with `x ^ x = 0`.

Do not infer more than the identity proves. XORing an array does not validate that every ordinary value appears exactly twice.

TRACE IT | 1.9 Worked dry run: decode a packed status byte

Suppose the low byte stores:

```text
bit 0: active
bit 1: verified
bit 2: premium
bit 3: suspended
```

The value is `0b0000_0101`.

```text
position: 3 2 1 0
value:    0 1 0 1
```

- active is true because bit 0 is one;
- verified is false because bit 1 is zero;
- premium is true because bit 2 is one; and
- suspended is false because bit 3 is zero.

Testing premium:

```text
value = 0101
mask  = 0100
AND   = 0100, which is nonzero
```

The correct test is `(value & mask) != 0`. Comparing with `== 1` works only for bit zero.

1.10 Common beginner mistakes

- Numbering positions from one instead of zero.
- Reading binary digits from left to right without labeling positions.
- Assuming an `int` has only the displayed significant bits.
- Using `1 << index` when the destination is a `long`.
- Using `>>` to inspect a raw negative bit pattern when `>>>` is needed.
- Treating `x << k` as safe multiplication for every input.
- Forgetting promotion when applying `~` to a `byte` or `short`.
- Comparing `(value & mask) == 1` instead of checking for nonzero.
- Memorizing a bit trick without its input contract.
- Calling a fixed 32-step loop `O(log n)` without explaining whether complexity is measured by numeric value or representation width.

1.11 Interview checks

Predict the output

JAVA

```java
System.out.println(12 & 10);
System.out.println(12 | 10);
System.out.println(12 ^ 10);
System.out.println(~5);
```

Expected output:

TEXT

```text
8
14
6
-6
```

Explain, do not only calculate

1. Why is `~0` equal to `-1` for an `int`?
2. Why can `1 << 31` be a valid mask even though it is negative?
3. When does `x >> 1` differ from `x / 2`?
4. Why is `1L << index` required for high `long` positions?
5. What contract would make an unsigned comparison necessary?

Debug the code

```java
static boolean isBitSet(long value, int index) {
    return (value & (1 << index)) == 1;
}
```

There are two defects: the mask is computed as an `int`, and comparing with one only recognizes bit zero. The corrected method also validates the index:

```java
static boolean isBitSet(long value, int index) {
    if (index < 0 || index >= Long.SIZE) {
        throw new IllegalArgumentException("index must be in [0, 63]");
    }
    return (value & (1L << index)) != 0;
}
```

RECAP | Chapter summary

- Bit positions start at zero from the right.
- Java bit operations occur over a fixed 32-bit or 64-bit representation.
- AND filters, OR sets or combines, XOR differs or toggles, and NOT inverts the whole width.
- `>>` extends the sign; `>>>` fills with zero.
- Java masks shift distances, so APIs must validate logical bit indexes.
- Small integral types promote to `int`.
- A suffix such as `L` must affect the operand before the shift.
- Always separate the bit pattern from its signed or unsigned interpretation.

TRY IT | Readiness checkpoint

Continue only when you can draw and explain all seven operators for small values, print a padded Java representation, and diagnose the `1 << 40` and `(value & mask) == 1` defects without notes.

SDE-2 CORE CHAPTER

Chapter 2 - Masks, Core Techniques, and Safe Shortcuts

The rule behind every mask

A mask marks positions that an operation should affect. The operator decides what happens to those positions:

- AND with a mask keeps selected positions;
- AND with an inverted mask clears selected positions;
- OR with a mask sets selected positions; and
- XOR with a mask toggles selected positions.

If you can derive those four actions from the truth tables, you do not need to memorize disconnected formulas.

Learning objectives

After this chapter, you should be able to:

- test, set, clear, toggle, and replace bits safely;
- build single-bit, low-bit, high-bit, and range masks;
- count, isolate, remove, and inspect set bits;
- recognize powers of two and four under precise contracts;
- calculate Hamming distance and required bit flips;
- reverse bits and compute per-value bit counts;
- choose Java library methods when they communicate intent better; and
- explain why each shortcut works and where it fails.

2.1 One-bit operations

Let `index` be a validated `int` position and let `mask = 1 << index`.

Goal	Expression	Reason
Test	`(value & mask) != 0`	AND retains the selected bit
Set	`value	mask`	
Clear	`value & ~mask`	inverted mask has zero at the selected bit
Toggle	`value ^ mask`	XOR with one flips a bit

Safe `long` helpers:

JAVA

```java
static long oneBit(int index) {
    if (index < 0 || index >= Long.SIZE) {
        throw new IllegalArgumentException("index must be in [0, 63]");
    }
    return 1L << index;
}

static boolean isSet(long value, int index) {
    return (value & oneBit(index)) != 0;
}

static long set(long value, int index) {
    return value | oneBit(index);
}

static long clear(long value, int index) {
    return value & ~oneBit(index);
}

static long toggle(long value, int index) {
    return value ^ oneBit(index);
}
```

TRACE IT | Dry run: clear bit 3 from 13

TEXT

```
value         = 1101
mask          = 1000
inverted mask = ...0111
result        = 0101 = 5
```

Only bit 3 changed. All other positions were ANDed with one and preserved.

2.2 Set a bit to a requested boolean value

A clear implementation is often best:

JAVA

```java
static long update(long value, int index, boolean enabled) {
    return enabled ? set(value, index) : clear(value, index);
}
```

An interviewer may ask for a branch-free version. Convert the boolean to either all zeros or all ones:

JAVA

```java
static long updateBranchFree(long value, int index, boolean enabled) {
    long mask = oneBit(index);
    long requested = enabled ? -1L : 0L;
    return (value & ~mask) | (requested & mask);
}
```

The branch-free form is not automatically faster in Java and is less obvious. Present the readable version first unless the constraint specifically rewards branch removal.

2.3 Build a mask for the lowest width bits

For widths from 1 through 63:

JAVA

```java
long mask = (1L << width) - 1;
```

Example for width 5:

TEXT

```
1L << 5       = 100000
(1L << 5) - 1 = 011111
```

Width 64 is a special case because Java masks the shift distance and `1L << 64` becomes `1L << 0`.

JAVA

```java
static long lowBitsMask(int width) {
    if (width < 0 || width > Long.SIZE) {
        throw new IllegalArgumentException("width must be in [0, 64]");
    }
    if (width == 0) {
        return 0L;
    }
    if (width == Long.SIZE) {
        return -1L;
    }
    return (1L << width) - 1;
}
```

2.4 Build and use a contiguous range mask

Suppose a field begins at `offset` and occupies `width` bits.

JAVA

```java
static long rangeMask(int offset, int width) {
    if (offset < 0 || width < 0 || offset > Long.SIZE - width) {
        throw new IllegalArgumentException("invalid range");
    }
    return lowBitsMask(width) << offset;
}
```

Extract an unsigned field:

JAVA

```java
static long extractField(long word, int offset, int width) {
    if (width == Long.SIZE) {
        if (offset != 0) {
            throw new IllegalArgumentException("invalid full-width field");
        }
        return word;
    }
    return (word >>> offset) & lowBitsMask(width);
}
```

Replace a field:

JAVA

```java
static long replaceField(long word, int offset, int width, long newValue) {
    long unshiftedMask = lowBitsMask(width);
    if ((newValue & ~unshiftedMask) != 0) {
        throw new IllegalArgumentException("new value does not fit");
    }
    long mask = unshiftedMask << offset;
    return (word & ~mask) | (newValue << offset);
}
```

The two phases are visible: clear the old field, then insert the new field. Validation prevents silent truncation.

2.5 The lowest set bit: x & -x

For nonzero x, $x \ \& \ -x$ isolates its lowest one bit.

TEXT

```
x            = 10110000
-x           = 01010000  (same illustrative width)
x & -x       = 00010000
```

Why it works:

1. `-x` is `~x + 1`.
2. Bits below the lowest one in `x` are zero.
3. The lowest one remains one in both `x` and `-x`.
4. Higher positions differ or are irrelevant to the AND result.

Use cases:

- partitioning two exceptional XOR values;
- Fenwick tree index movement;
- iterating selected positions; and
- converting a set bit into its position with `numberOfTrailingZeros`.

JAVA

```java
int lowestMask = value & -value;
int index = Integer.numberOfTrailingZeros(value); // 32 when value is zero
```

`Integer.numberOfTrailingZeros(0)` returning 32 is a defined sentinel, not a valid `int` bit index. Check zero when the contract requires an actual selected bit.

2.6 Remove the lowest set bit: x & (x - 1)

TEXT

```
x       = 10110000
x - 1   = 10101111
AND     = 10100000
```

Subtracting one clears the lowest one and changes lower zeros to ones. AND removes that lowest one and clears the changed suffix.

Brian Kernighan's set-bit count

JAVA

```java
static int countSetBits(int value) {
    int count = 0;
    while (value != 0) {
        value &= value - 1;
        count++;
    }
    return count;
}
```

Loop invariant: `count` equals the number of one bits removed from the original value, and `value` contains exactly the remaining one bits.

Time is `O(p)`, where `p` is the number of set bits. Since an `int` has 32 positions, this is bounded by 32 iterations. Stating `O(p)` explains the progress measure; stating constant time under a fixed-width machine model explains the bound.

Iterate only the set positions

JAVA

```java
for (int remaining = value; remaining != 0; remaining &= remaining - 1) {
    int index = Integer.numberOfTrailingZeros(remaining);
    // process index
}
```

This visits selected positions without scanning all 32. It is useful when the mask is sparse.

2.7 Power-of-two recognition

A positive power of two has exactly one set bit.

JAVA

```java
static boolean isPowerOfTwo(int value) {
    return value > 0 && (value & (value - 1)) == 0;
}
```

The positivity check is mandatory. Without it:

- zero passes because `0 & -1` is zero; and
- `Integer.MIN_VALUE` passes the one-bit test even though the usual interview contract asks for a positive power of two.

Java already provides `Integer.bitCount(value) == 1`, but it needs the same positive-value contract.

Power of four

A positive power of four has one set bit in an even index: 0, 2, 4, and so on.

JAVA

```java
static boolean isPowerOfFour(int value) {
    return value > 0
            && (value & (value - 1)) == 0
            && (value & 0x5555_5555) != 0;
}
```

`0x5555_5555` is the repeating pattern `0101...0101`, selecting even positions. An arithmetic alternative is `value % 3 == 1` after the power-of-two check, because powers of four are `1 mod 3`; the mask version reveals the bit position directly.

2.8 Hamming distance and minimum flips

The Hamming distance between two fixed-width integers is the number of positions on which they differ.

JAVA

```
static int hammingDistance(int first, int second) {
    return Integer.bitCount(first ^ second);
}
```

XOR creates one exactly where the operands differ; bit count finishes the task.

Flips needed so (a | b) == target

For each position:

- if target is zero, every one in `a` or `b` must be cleared;
- if target is one and both inputs are zero, one input must be set; and
- otherwise no change is needed.

JAVA

```
static int flipsForOr(int a, int b, int target) {
    int flips = 0;
    for (int bit = 0; bit < Integer.SIZE; bit++) {
        int aBit = (a >>> bit) & 1;
        int bBit = (b >>> bit) & 1;
        int targetBit = (target >>> bit) & 1;
        if (targetBit == 0) {
            flips += aBit + bBit;
        } else if (aBit == 0 && bBit == 0) {
            flips++;
        }
    }
    return flips;
}
```

The per-bit truth table is clearer than attempting a memorized combined expression during an interview.

2.9 Reverse a 32-bit representation

Build the result from left to right while consuming the input from right to left.

JAVA

```
static int reverseBits(int value) {
    int reversed = 0;
    for (int bit = 0; bit < Integer.SIZE; bit++) {
        reversed = (reversed << 1) | (value & 1);
        value >>>= 1;
    }
    return reversed;
}
```

Use `>>>` so a negative input eventually shifts in zeros. The loop must run exactly 32 times because leading zeros in the input become trailing zeros in the result and still belong to the fixed-width contract.

Java provides `Integer.reverse(value)` and `Long.reverse(value)`. Know the derivation; use the library when the interview allows it.

2.10 Count bits for every value from zero through n

Removing the lowest set bit gives a dynamic-programming relation:

TEXT

```
bits[x] = bits[x & (x - 1)] + 1
```

The predecessor is smaller than `x`, so it has already been computed.

JAVA

```java
static int[] countBitsThrough(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("n must be nonnegative");
    }
    int[] bits = new int[n + 1];
    for (int value = 1; value <= n; value++) {
        bits[value] = bits[value & (value - 1)] + 1;
    }
    return bits;
}
```

Time is `O(n)` and output space is `O(n)`. The recurrence saves repeated per-value scanning; it does not reduce the requirement to produce `n + 1` answers.

An equally valid relation is:

TEXT

```
bits[x] = bits[x >>> 1] + (x & 1)
```

Choose the one you can explain most clearly.

2.11 Highest bit, bit length, and next capacity

Java exposes useful operations:

Goal	Java API
count ones	`Integer.bitCount(x)`
lowest one-bit mask	`Integer.lowestOneBit(x)`
highest one-bit mask	`Integer.highestOneBit(x)`
leading zeros	`Integer.numberOfLeadingZeros(x)`
trailing zeros	`Integer.numberOfTrailingZeros(x)`
rotate	`Integer.rotateLeft(x, distance)`
reverse bits	`Integer.reverse(x)`
unsigned string	`Integer.toUnsignedString(x)`

For positive `x`, its bit length is:

JAVA

```java
int bitLength = Integer.SIZE - Integer.numberOfLeadingZeros(x);
```

For zero, define the bit length as zero explicitly.

A common capacity problem asks for the smallest power of two at least as large as a positive value. Derive it carefully and handle overflow rather than copying a compact formula. For an `int`, the next positive power of two cannot exceed `1 << 30`.

JAVA

```java
static int ceilingPowerOfTwo(int value) {
    if (value <= 1) {
        return 1;
    }
    if (value > (1 << 30)) {
        throw new ArithmeticException("result does not fit positive int");
    }
    return 1 << (Integer.SIZE - Integer.numberOfLeadingZeros(value - 1));
}
```

Subtracting one makes an existing power of two remain unchanged after rounding.

2.12 A shortcut selection table

Problem wording	Candidate technique	Contract to say aloud
Is this a positive power of two?	`x > 0 && (x & (x - 1)) == 0`	positive mathematical value
Count selected positions	Kernighan or `bitCount`	count over 32 or 64 representation bits
Find the lowest selected position	`x & -x` or trailing zeros	input must be nonzero for a valid position
Remove one selected position	`x &= x - 1`	removes the lowest one only
Compare bit patterns	`bitCount(a ^ b)`	fixed width and sign interpretation defined
Read an unsigned field	`>>>` then AND	offset and width validated
Force selected bits on	OR with mask	mask width matches value width
Force selected bits off	AND with inverted mask	inversion spans full width
Flip selected bits	XOR with mask	toggling twice restores original

The table is a retrieval aid. During an interview, derive the expression from the desired per-position behavior and then state the precondition.

2.13 Common failures and corrections

Failure: check whether the third bit is set

JAVA

```java
return (value & (1 << 2)) == 1;
```

Correction:

JAVA

```java
return (value & (1 << 2)) != 0;
```

Failure: clear with XOR

JAVA

```java
value ^= mask;
```

XOR toggles. If the bit was zero, this sets it. To clear unconditionally:

JAVA

```java
value &= ~mask;
```

Failure: power-of-two test accepts zero

```java
return (value & (value - 1)) == 0;
```

Correction:

```java
return value > 0 && (value & (value - 1)) == 0;
```

Failure: build all-low-bits mask for width 64

```java
return (1L << width) - 1;
```

Handle width zero and width 64 explicitly.

2.14 Interview follow-ups

1. Why can `x & -x` be negative yet still be a correct mask?
2. Compare Kernighan's loop with scanning all 32 positions.
3. What does `Integer.bitCount(-1)` return, and why?
4. Why is a branch-free update not automatically faster on the JVM?
5. How would you generalize these operations to more than 64 flags?
6. Why must a packed-field replacement validate the new value before ORing it?
7. What is the time complexity of bit count if input size is measured in bits rather than as one machine word?

RECAP | Chapter summary

- A mask plus an operator gives a precise per-position update.
- `x & -x` isolates the lowest set bit; `x & (x - 1)` removes it.
- Power-of-two tests need an explicit positive-input condition.
- XOR plus bit count computes Hamming distance.
- Range masks need special handling at widths zero and 64.
- Java's named bit APIs are often the clearest production choice.
- Safe interview shortcuts include their derivation, fixed width, and input contract.

TRY IT | Readiness checkpoint

From memory, implement test, set, clear, toggle, low-bit isolation, set-bit count, positive power-of-two detection, Hamming distance, and field extraction. For each, explain one failing edge case in the naive version.

SDE-2 CORE CHAPTER

Chapter 3 - XOR Patterns and Prefix State

Begin with the contract, not the trick

XOR is powerful because equal bit patterns cancel:

```text
x ^ x = 0
x ^ 0 = x
```

It is also associative and commutative, so the order and grouping of a reduction do not matter. These facts solve a problem only when the occurrence counts or prefix structure match them.

Before choosing XOR, state:

- which values are repeated;
- how many times they repeat;
- how many exceptional values exist;
- whether input values are fixed-width Java integers; and
- whether an answer must be interpreted using signed or unsigned order.

Learning objectives

After this chapter, you should be able to:

- prove pair cancellation with a loop invariant;
- solve one-single, two-single, and triple-occurrence families;
- use XOR to find a missing value under a precise range contract;
- derive prefix XOR for range queries;
- use the four-value cycle for XOR from zero through `n`;
- count subarrays whose XOR equals a target;
- recognize when XOR cannot validate the input promise; and
- avoid swap, signed-order, and duplicate-contract traps.

WHY IT WORKS | 3.1 The cancellation invariant

Consider:

JAVA

```java
int xor = 0;
for (int value : values) {
    xor ^= value;
}
```

Loop invariant: after processing the first `i` elements, `xor` equals the XOR of exactly those elements.

At termination, algebra may simplify the reduction. If every ordinary value occurs twice and one value occurs once, every pair becomes zero and the exceptional value remains.

XOR does not prove the occurrence contract. For example, four copies also cancel, and three copies reduce to one copy. If validation is required, use counting or a set even though it costs extra space.

3.2 Single Number I: one value among pairs

Contract: the array is nonempty, exactly one value appears once, and every other value appears exactly twice.

JAVA

```java
static int singleAmongPairs(int[] values) {
    if (values == null || values.length == 0) {
        throw new IllegalArgumentException("values must not be empty");
    }
    int answer = 0;
    for (int value : values) {
        answer ^= value;
    }
    return answer;
}
```

Dry run for `[4, 1, 2, 1, 2]`:

Read	Accumulator
start	`0`
`4`	`4`
`1`	`4 ^ 1 = 5`
`2`	`5 ^ 2 = 7`
`1`	`7 ^ 1 = 6`
`2`	`6 ^ 2 = 4`

Time is `O(n)` and auxiliary space is `O(1)`. A `HashSet` alternative can validate and generalize counts, but it uses `O(n)` space.

Interview follow-up: what if values appear four times?

Four identical values cancel too. The code still returns an XOR result, but the original semantic promise has changed. Say whether counts divisible by two are acceptable or whether exactly twice must be validated.

3.3 Missing value from the range 0 through n

Contract: an array of length `n` contains distinct values from `0..n` with exactly one missing.

XOR every expected value and every observed value. Present pairs cancel.

JAVA

```java
static int missingFromZeroThroughN(int[] values) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
    int answer = values.length;
    for (int index = 0; index < values.length; index++) {
        answer ^= index;
        answer ^= values[index];
    }
    return answer;
}
```

This avoids the overflow risk of summing `0..n`, but it does not validate distinctness or range membership. A malformed array may produce a plausible answer.

Complexity is `O(n)` time and `O(1)` auxiliary space.

3.4 Single Number III: two values among pairs

Contract: exactly two distinct values appear once; every other value appears twice.

Let the exceptional values be `a` and `b`.

1. XOR everything: `combined = a ^ b`.
2. Since `a != b`, `combined` is nonzero.
3. Select one set bit in `combined`; `a` and `b` differ there.
4. Partition every input by that bit.
5. Pairs remain in the same partition and cancel.

JAVA

```java
static int[] twoSinglesAmongPairs(int[] values) {
    if (values == null || values.length < 2) {
        throw new IllegalArgumentException("at least two values required");
    }
    int combined = 0;
    for (int value : values) {
        combined ^= value;
    }
    if (combined == 0) {
        throw new IllegalArgumentException("two distinct singles required");
    }

    int distinguishingBit = combined & -combined;
    int first = 0;
    int second = 0;
    for (int value : values) {
        if ((value & distinguishingBit) == 0) {
            first ^= value;
        } else {
            second ^= value;
        }
    }
    return first <= second
            ? new int[] {first, second}
            : new int[] {second, first};
}
```

TRACE IT | Dry run

Input: `[1, 2, 1, 3, 2, 5]`

TEXT

```
combined = 3 ^ 5 = 0110
lowest selected mask = 0010

mask clear group: 1, 1, 5 -> 5
mask set group:   2, 3, 2 -> 3
```

The selected mask may be `Integer.MIN_VALUE`. That is valid: it represents the sign-bit position on which the exceptional values differ. Do not call `Math.abs` on it.

3.5 Single Number II: one value among triples

Contract: one value occurs once and every other value occurs exactly three times.

Clear baseline: count each bit modulo three

JAVA

```java
static int singleAmongTriplesByCount(int[] values) {
    if (values == null || values.length == 0) {
        throw new IllegalArgumentException("values must not be empty");
    }
    int answer = 0;
    for (int bit = 0; bit < Integer.SIZE; bit++) {
        int count = 0;
        for (int value : values) {
            count += (value >>> bit) & 1;
        }
        if (count % 3 != 0) {
            answer |= 1 << bit;
        }
    }
    return answer;
}
```

Bit 31 is reconstructed just like every other bit, so negative exceptional values work.

Time is `O(32n)` and auxiliary space is `O(1)`. Under a fixed-width model, this is `O(n)`, but `O(32n)` shows the actual structure.

Compact state-machine version

For every bit independently, track whether its count is one or two modulo three:

TEXT

```
count state: 0 -> 1 -> 2 -> 0
mask state: 00 -> 01 -> 10 -> 00
```

JAVA

```java
static int singleAmongTriples(int[] values) {
    int ones = 0;
    int twos = 0;
    for (int value : values) {
        ones = (ones ^ value) & ~twos;
        twos = (twos ^ value) & ~ones;
    }
    return ones;
}
```

Invariant: after each input, no bit is present in both `ones` and `twos`; their two-bit state represents the count modulo three.

During an interview, begin with per-bit counts unless you can derive the state machine confidently. A compact answer with no explanation is weaker than a clear correct baseline.

3.6 Prefix XOR for repeated range queries

Prefix sums subtract a repeated prefix. Prefix XOR cancels it.

Define a half-open prefix:

```text
prefix[0] = 0
prefix[i + 1] = values[0] ^ ... ^ values[i]
```

Then the inclusive range `[left, right]` is:

```text
prefix[right + 1] ^ prefix[left]
```

Every value before `left` appears twice and cancels.

```java
static int[] buildPrefixXor(int[] values) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
    int[] prefix = new int[values.length + 1];
    for (int index = 0; index < values.length; index++) {
        prefix[index + 1] = prefix[index] ^ values[index];
    }
    return prefix;
}

static int rangeXor(int[] prefix, int left, int right) {
    if (prefix == null || left < 0 || right < left
            || right + 1 >= prefix.length) {
        throw new IllegalArgumentException("invalid range");
    }
    return prefix[right + 1] ^ prefix[left];
}
```

Build: `O(n)` time and `O(n)` space. Query: `O(1)` time.

For one query, a direct loop may be simpler and avoids prefix storage. Prefix state pays off when the array is unchanged and there are many queries.

TRACE IT | Dry run

Values: `[4, 2, 7, 2]`

Prefix length	Prefix XOR
0	0
1	4
2	`4 ^ 2 = 6`
3	`6 ^ 7 = 1`
4	`1 ^ 2 = 3`

Range `[1, 3]` is `prefix[4] ^ prefix[1] = 3 ^ 4 = 7`, equal to `2 ^ 7 ^ 2`.

3.7 XOR from zero through n in constant time

For nonnegative `n`, the XOR `0 ^ 1 ^ ... ^ n` repeats by `n mod 4`:

`n & 3`	Result
0	`n`
1	`1`
2	`n + 1`
3	`0`

JAVA

```java
static int xorZeroThrough(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("n must be nonnegative");
    }
    return switch (n & 3) {
        case 0 -> n;
        case 1 -> 1;
        case 2 -> n + 1;
        default -> 0;
    };
}
```

Derivation

Group four consecutive values. For `4k` through `4k + 3`, the XOR is zero. The incomplete tail determines the result. Verify the first eight prefix results:

TEXT

```
n:      0 1 2 3 4 5 6 7
prefix: 0 1 3 0 4 1 7 0
```

Inclusive range `[left, right]` for nonnegative integers is:

JAVA

```java
static int xorRange(int left, int right) {
    if (left < 0 || right < left) {
        throw new IllegalArgumentException("invalid nonnegative range");
    }
    return xorZeroThrough(right)
            ^ (left == 0 ? 0 : xorZeroThrough(left - 1));
}
```

State the nonnegative-domain precondition. Do not casually extend this cycle across signed overflow.

3.8 Count subarrays whose XOR equals a target

This is the XOR analogue of counting subarrays with a target sum.

Let `prefix` be the XOR through the current position. A prior prefix `previous` creates target XOR when:

TEXT

```
previous ^ prefix = target
previous = prefix ^ target
```

Store how often each prior prefix has appeared.

JAVA

```java
import java.util.HashMap;
import java.util.Map;

static long countSubarraysWithXor(int[] values, int target) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
    Map<Integer, Integer> frequency = new HashMap<>();
    frequency.put(0, 1);
    int prefix = 0;
    long count = 0;
    for (int value : values) {
        prefix ^= value;
        count += frequency.getOrDefault(prefix ^ target, 0);
        frequency.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

The initial frequency of zero counts subarrays starting at index zero. Use `long` for the number of subarrays because there can be `n(n + 1)/2` matches.

Expected time is `O(n)` with `HashMap`; auxiliary space is `O(n)`. If hash behavior or adversarial keys matter, qualify the cost model rather than claiming an unconditional guarantee.

Invariant

Before processing the next element, the map contains frequencies of all prefix XORs ending before that element. After updating `prefix`, every matching prior prefix identifies one distinct subarray ending at the current position.

3.9 XOR swap: a trick to recognize, not recommend

JAVA

```java
first ^= second;
second ^= first;
first ^= second;
```

This swaps two distinct variables without a temporary value. It is inferior interview and production Java in most situations:

- it is harder to read;
- it fails conceptually when both names refer to the same storage location, such as swapping an array element with itself;
- the JVM can optimize an ordinary temporary-variable swap; and
- it distracts from the actual algorithm.

Use:

```java
int temporary = first;
first = second;
second = temporary;
```

Knowing the XOR version helps answer a trivia follow-up. Choosing clarity demonstrates better engineering judgment.

CHOOSE IT | 3.10 XOR decision map

Input promise or query	Pattern	Main warning
one value once, all others twice	XOR reduction	does not validate counts
two values once, all others twice	combined XOR plus partition	combined must be nonzero
one value once, all others three times	per-bit modulo count	include sign bit
one value missing from distinct `0..n`	expected XOR observed	validate range only if required
many immutable range-XOR queries	prefix XOR	define endpoints consistently
XOR of integer range	four-value cycle	nonnegative domain
count subarrays with target XOR	prefix-frequency map	initialize zero prefix
maximize XOR partner	high-to-low trie	signed versus unsigned objective

3.11 Predict, debug, and explain

Predict

```java
int x = 9;
System.out.println(x ^ x);
System.out.println(x ^ 0);
System.out.println(x ^ -1);
```

Output:

```text
0
9
-10
```

`x ^ -1` flips every position and therefore equals `~x`.

Debug

JAVA

```java
static int rangeXor(int[] prefix, int left, int right) {
    return prefix[right] ^ prefix[left];
}
```

With a half-open prefix array, the correct inclusive query is `prefix[right + 1] ^ prefix[left]`. Name the prefix convention before writing the formula.

Explain

1. Why does the two-single partition keep every duplicate pair together?
2. Why does XOR avoid arithmetic overflow in the missing-number problem?
3. Why can malformed input still produce a plausible XOR answer?
4. Why is `long` appropriate for a subarray count even though prefix state is `int`?
5. What changes if the objective compares XOR values as unsigned integers?

RECAP | Chapter summary

- XOR cancellation is algebra applied under an occurrence contract.
- One exceptional value needs one reduction; two need a distinguishing-bit partition.
- Triple occurrences can be handled with per-bit counts modulo three before attempting a compact state machine.
- Prefix XOR gives constant-time immutable range queries.
- Prefix frequency counts target-XOR subarrays in expected linear time.
- The `0..n` XOR cycle is a derived four-case pattern for nonnegative ranges.
- XOR swap is interview trivia, not preferred Java style.

TRY IT | Readiness checkpoint

Without notes, solve the one-single, two-single, missing-number, range-XOR, and target-subarray-XOR problems. State the exact input promise, loop invariant, time, auxiliary space, and one malformed-input limitation for each.

SDE-2 CORE CHAPTER

Chapter 4 - Subsets, Submasks, and Compact State

Why masks represent subsets

For `n` indexed items, an `n`-bit value can record whether each item is absent or present. Bit `i` corresponds to item `i`.

For items `[A, B, C]`:

Mask	Selected items
`000`	none
`001`	A
`010`	B
`011`	A, B
`100`	C
`101`	A, C
`110`	B, C
`111`	A, B, C

This is not merely a storage trick. It turns membership into integer state, enabling fast checks, compact memoization keys, subset enumeration, and submask transitions.

Learning objectives

After this chapter, you should be able to:

- map indexed items to bit positions;
- enumerate all subsets without hiding exponential output;
- distinguish index subsets from distinct value subsets;
- enumerate all submasks of one mask;
- derive the `O(3^n)` total mask-submask bound;
- generate Gray-code order and identify its limited benefit;
- use masks as small-state keys for search or dynamic programming; and
- reject mask representations that do not fit the constraints.

4.1 Enumerate every subset

For `n` items, masks range from zero through `2^n - 1`.

JAVA

```java
static List<List<Integer>> subsets(int[] values) {
    if (values == null || values.length > 20) {
        throw new IllegalArgumentException("0..20 items required");
    }
    int total = 1 << values.length;
    List<List<Integer>> result = new ArrayList<>(total);
    for (int mask = 0; mask < total; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int index = 0; index < values.length; index++) {
            if ((mask & (1 << index)) != 0) {
                subset.add(values[index]);
            }
        }
        result.add(List.copyOf(subset));
    }
    return List.copyOf(result);
}
```

The limit of 20 is an API decision for materialized output, not a universal threshold. At `n = 20`, there are already 1,048,576 subsets.

TRACE IT | Dry run for [10, 20, 30]

For mask `101`:

TEXT

```text
bit 0 is 1 -> include 10
bit 1 is 0 -> skip 20
bit 2 is 1 -> include 30
result: [10, 30]
```

Complexity

There are `2^n` masks and the baseline checks `n` positions per mask:

```text
time = Theta(n * 2^n)
```

If all subsets are materialized, the output itself contains `Theta(n * 2^n)` element occurrences across all lists. Calling this `O(2^n)` ignores the cost of producing each subset.

Auxiliary working space can be `O(n)` when subsets are streamed to a consumer. Stored result space is output-sized.

4.2 Index subsets versus distinct value subsets

If the input is `[2, 2]`, masks `01` and `10` select different indexes but produce the same value list `[2]`.

Bitmask enumeration naturally generates index subsets. If the problem requires unique value combinations, you need an additional strategy:

- sort and skip equal choices during backtracking;
- store generated subsets in a set, usually less efficient; or
- compress value frequencies and enumerate counts.

Do not claim the bitmask loop removes duplicates. Define what a unique subset means in the problem contract.

4.3 Stream instead of materialize

When the task only evaluates each subset, avoid storing all of them.

JAVA

```java
static long maximumSubsetSumAtMost(int[] values, long limit) {
    if (values == null || values.length > 25) {
        throw new IllegalArgumentException("0..25 items required");
    }
    long best = 0;
    long total = 1L << values.length;
    for (long mask = 0; mask < total; mask++) {
        long sum = 0;
        for (int index = 0; index < values.length; index++) {
            if ((mask & (1L << index)) != 0) {
                sum += values[index];
            }
        }
        if (sum <= limit) {
            best = Math.max(best, sum);
        }
    }
    return best;
}
```

This changes storage from output-sized to `O(1)` working state, but time remains exponential. It is useful only for small `n`, and negative values may change pruning or initialization choices.

4.4 Enumerate the selected positions efficiently

Instead of scanning all `n` positions, iterate only the set bits:

JAVA

```java
for (int remaining = mask; remaining != 0; remaining &= remaining - 1) {
    int index = Integer.numberOfTrailingZeros(remaining);
    // process item at index
}
```

For one subset with `k` selected items, this performs `O(k)` iterations. Across all subsets, the total number of selected-item visits is exactly `n * 2^(n - 1)`, because each item appears in half of all subsets. The overall output-sensitive bound remains `Theta(n * 2^n)`.

4.5 Enumerate every submask of one mask

A submask contains only positions selected by the original mask.

JAVA

```java
static List<Integer> nonzeroSubmasks(int mask) {
    List<Integer> result = new ArrayList<>();
    for (int sub = mask; sub != 0; sub = (sub - 1) & mask) {
        result.add(sub);
    }
    return result;
}
```

For `mask = 10110`:

TEXT

```
10110 -> 10100 -> 10010 -> 10000
      -> 00110 -> 00100 -> 00010 -> stop
```

If zero is valid, handle it after the loop:

JAVA

```java
// process zero submask here
```

Why (sub - 1) & mask works

`sub - 1` moves to a lower pattern, possibly turning on positions outside the original mask. AND removes those forbidden positions. Repeating produces the next lower allowed pattern.

For a mask with `k` selected positions, there are `2^k` submasks including zero.

The zero-loop trap

This form never terminates:

JAVA

```java
for (int sub = mask; ; sub = (sub - 1) & mask) {
    // after sub becomes zero, the update returns mask again
}
```

Use `sub != 0` and process zero separately, or break explicitly after processing zero.

4.6 Why all masks with all submasks cost O(3^n)

Consider:

```java
for (int mask = 0; mask < (1 << n); mask++) {
    for (int sub = mask; sub != 0; sub = (sub - 1) & mask) {
        // transition from sub to mask
    }
}
```

Each bit position has three roles:

1. outside `mask`;
2. inside `mask` but outside `sub`; or
3. inside both `mask` and `sub`.

That gives `3^n` mask-submask relationships, excluding or including a lower-order zero case depending on the loop. The bound is not `4^n`; `sub` cannot contain a position outside `mask`.

This derivation is a common SDE-2 follow-up because it tests combinatorial reasoning, not syntax.

4.7 Complement within an n-bit universe

`~mask` flips all 32 positions, not only the `n` logical positions.

Wrong for an `n`-item universe:

```java
int complement = ~mask;
```

Correct:

```java
int universe = (1 << n) - 1;
int complement = universe ^ mask;
```

For `n == 32`, the expression `1 << n` fails because the distance is masked. A robust API either uses a `long` for up to 31 or 32 logical items, or handles the full-width case explicitly.

4.8 Gray code: change one selected item at a time

The Gray code for sequence number `i` is:

TEXT

```
gray(i) = i ^ (i >>> 1)
```

Consecutive Gray codes differ in exactly one bit.

JAVA

```java
static int grayCode(int index) {
    if (index < 0) {
        throw new IllegalArgumentException("index must be nonnegative");
    }
    return index ^ (index >>> 1);
}
```

First eight values:

TEXT

```
i:       000 001 010 011 100 101 110 111
gray(i): 000 001 011 010 110 111 101 100
```

To find the changed position between consecutive masks:

JAVA

```java
int changed = previous ^ current;
int index = Integer.numberOfTrailingZeros(changed);
```

Gray order helps only if an aggregate can be updated from the one changed item. If each subset still requires a full scan, Gray code does not improve the asymptotic time and may reduce clarity.

4.9 Mask as visited-state key

For a small bounded set of tasks, cities, keys, or workers, a mask can encode which items have been used.

Example: assign one distinct job to each worker. `mask` records assigned jobs; `worker = bitCount(mask)` identifies the next worker.

JAVA

```java
static int minimumAssignmentCost(int[][] cost) {
    int n = cost.length;
    if (n == 0 || n > 20) {
        throw new IllegalArgumentException("1..20 workers required");
    }
    for (int[] row : cost) {
        if (row == null || row.length != n) {
            throw new IllegalArgumentException("cost must be square");
        }
    }

    int total = 1 << n;
    long[] best = new long[total];
    Arrays.fill(best, Long.MAX_VALUE);
    best[0] = 0;

    for (int mask = 0; mask < total; mask++) {
        if (best[mask] == Long.MAX_VALUE) {
            continue;
        }
        int worker = Integer.bitCount(mask);
        if (worker == n) {
            continue;
        }
        for (int job = 0; job < n; job++) {
            int jobBit = 1 << job;
            if ((mask & jobBit) == 0) {
                int next = mask | jobBit;
                best[next] = Math.min(best[next], best[mask] + cost[worker][job]);
            }
        }
    }
    return Math.toIntExact(best[total - 1]);
}
```

State definition: `best[mask]` is the minimum cost to assign the first `bitCount(mask)` workers to exactly the jobs selected in `mask`.

Time is `O(n * 2^n)` and space is `O(2^n)`. This is feasible only for small `n`. The Dynamic Programming volume develops state design, reconstruction, and compression more deeply; here the goal is to understand why a mask can replace a set in the state key.

4.10 Recognition versus overuse

Choose a primitive mask when:

- the universe is small and fixed;
- membership is boolean;
- indexes are stable and dense;
- state copying or hashing would otherwise be frequent; or
- a subset-state algorithm already has exponential constraints.

Do not choose a primitive mask when:

- identifiers are sparse or unbounded;
- more than 64 logical positions are required and no segmented design exists;
- readability of named values matters more than compactness;
- the mapping from items to positions is unstable; or
- the underlying search is infeasible even with compact state.

A mask improves representation cost. It does not make an exponential state space polynomial.

4.11 Common interview traps

- `1 << n` overflows or wraps before subset enumeration.
- A `long` mask does not make `2^60` work feasible.
- Input duplicates create duplicate value subsets even when masks are unique.
- `~mask` includes bits outside the logical universe.
- Submask enumeration can cycle after zero.
- Materialized-output space is incorrectly reported as `O(1)`.
- The DP state omits information needed for future choices.
- `bitCount(mask)` is used as a step number when not every transition adds exactly one item.
- A mask is used for external identifiers whose numeric values are not dense bit indexes.
- Gray code is introduced without an incremental update that benefits from it.

4.12 Interview follow-ups

1. Derive the `Theta(n * 2^n)` total size of all subsets.
2. Why does the mask-submask pair count become `3^n`?
3. How would you generate unique subsets from duplicate input values?
4. When is backtracking clearer than integer-mask enumeration?
5. How would you represent 100 possible flags?
6. Can a mask be used as a `HashMap` key safely? What makes primitive wrapper keys immutable?
7. When does meet-in-the-middle reduce a `2^n` search to roughly `2^(n/2)` work per side?

The final question is an advanced direction, not a default solution. Meet-in-the-middle is covered with array and search techniques later in the series.

RECAP | Chapter summary

- An `n`-bit mask naturally represents one subset of `n` indexed items.
- Full subset generation has unavoidable exponential output.
- Index uniqueness and value uniqueness are different contracts.
- `(sub - 1) & mask` enumerates allowed submasks; zero needs deliberate handling.
- All mask-submask relationships across `n` positions total `3^n`.
- Gray code changes one position at a time but helps only with incremental state.
- Bitmask state can replace a small immutable set in search or DP.
- Compact representation does not rescue an infeasible state space.

TRY IT | Readiness checkpoint

Implement subset and submask enumeration, explain their output-sensitive complexity, diagnose the zero-loop trap, and define one valid `dp[mask]` invariant before continuing to advanced patterns.

SDE-2 CORE CHAPTER

Chapter 5 - SDE-2 Bit Interview Patterns

Use this chapter after the foundations

The earlier chapters established representation, masks, XOR, and subset state. This chapter combines those tools into frequently asked interview patterns. Each technique is presented as:

1. recognition signal;
2. baseline;
3. key invariant or monotonic property;
4. optimized Java implementation;
5. boundary cases; and
6. SDE-2 follow-up.

Do not force these techniques onto every integer problem. The best answer begins with constraints and a clear baseline.

Learning objectives

After this chapter, you should be able to:

- maximize XOR with a high-to-low trie;
- compute range AND by finding a common binary prefix;
- count all set bits from one through `n` by block structure;
- compute a significant-bit complement;
- prove why the minimum XOR pair is adjacent after sorting;
- use bounded distinct OR states for subarray reasoning;
- add integers with bit operations under Java overflow semantics; and
- compare bit solutions with simpler sorting, hashing, and library alternatives.

5.1 Maximum XOR pair with a bitwise trie

Recognition signal

The problem asks for the maximum `a ^ b` among many values or repeated maximum-XOR queries.

Baseline

Compare all pairs: `O(n^2)` time and `O(1)` space.

Greedy bit insight

At the highest relevant bit, a differing pair produces one. That high one outweighs every possible combination of lower bits. Therefore, when querying for `value`, prefer a trie branch containing the opposite bit at each position.

For nonnegative `int` values, bits 30 through 0 determine ordinary signed order. If negative values are allowed, clarify whether the objective uses signed or unsigned comparison.

Trie implementation for nonnegative inputs

JAVA (continued 1/2)

```java
static final class BitNode {
    final BitNode[] next = new BitNode[2];
}

static int maximumXorPair(int[] values) {
    if (values == null || values.length < 2) {
        throw new IllegalArgumentException("at least two values required");
    }
    BitNode root = new BitNode();
    for (int value : values) {
        if (value < 0) {
            throw new IllegalArgumentException("nonnegative values required");
        }
        insert(root, value);
    }

    int best = 0;
    for (int value : values) {
        best = Math.max(best, bestPartnerXor(root, value));
    }
    return best;
}

static void insert(BitNode root, int value) {
    BitNode node = root;
```

JAVA (continued 2/2)

```
    for (int bit = 30; bit >= 0; bit--) {
        int current = (value >>> bit) & 1;
        if (node.next[current] == null) {
            node.next[current] = new BitNode();
        }
        node = node.next[current];
    }
}

static int bestPartnerXor(BitNode root, int value) {
    BitNode node = root;
    int result = 0;
    for (int bit = 30; bit >= 0; bit--) {
        int current = (value >>> bit) & 1;
        int preferred = current ^ 1;
        if (node.next[preferred] != null) {
            result |= 1 << bit;
            node = node.next[preferred];
        } else {
            node = node.next[current];
        }
    }
    return result;
}
```

For width `w`, time is `O(nw)` and worst-case node space is `O(nw)`. With 31 fixed positions this is often summarized as linear, but keeping `w` visible explains the memory cost.

SDE-2 follow-ups

- For online insert-and-query operations, insert incrementally and query before or after insertion according to whether self-pairing is allowed.
- For unsigned 32-bit maximization, include bit 31 and compare candidates with `Integer.compareUnsigned`.
- A node-per-bit object trie creates allocation overhead. Production implementations may use primitive arrays of child indexes.
- For static data and only a few values, the quadratic baseline may be simpler and faster in practice.

5.2 Maximum XOR under a limit

A common follow-up asks: for each query `(x, limit)`, maximize `x ^ value` using only stored values `<= limit`.

Technique:

1. sort input values;
2. sort queries by `limit`, remembering original positions;
3. insert eligible values into the trie as limits increase; and
4. query greedily.

This offline ordering turns repeated filtering into one monotonic scan. Complexity is `O((n + q) log(n + q) + (n + q)w)`. The Arrays and Sorting modules develop offline-query design further; the bit-specific part is the trie query.

5.3 Bitwise AND of every value in an inclusive range

Recognition signal

Compute `left & (left + 1) & ... & right` without visiting the entire range.

Baseline

Loop across the range. This can be enormous and may overflow the loop variable near `Integer.MAX_VALUE`.

Common-prefix insight

Any bit that changes between `left` and `right` becomes zero somewhere in the range. Only the common high prefix survives.

JAVA

```java
static int rangeBitwiseAnd(int left, int right) {
    if (left < 0 || right < left) {
        throw new IllegalArgumentException("0 <= left <= right required");
    }
    int shifts = 0;
    while (left != right) {
        left >>>= 1;
        right >>>= 1;
        shifts++;
    }
    return left << shifts;
}
```

Dry run for `[26, 30]`:

```text
26 = 11010
30 = 11110
common prefix is 11
remaining positions become zero
answer = 11000 = 24
```

The loop runs at most 31 times for the nonnegative `int` contract, so it is `O(w)`.

Alternative: clear right's lowest one

```java
while (left < right) {
    right &= right - 1;
}
return right;
```

Each iteration removes a suffix-changing one bit from the upper bound. Use the common-prefix explanation first because its proof is easier to communicate.

5.4 Count all set bits from one through n

Recognition signal

Return the total number of one bits in every nonnegative integer from `1` to `n`, where looping through all numbers is too slow.

Highest-power block insight

Let `2^k` be the highest power of two not greater than `n`.

1. In values `0..2^k - 1`, each of the `k` positions is one exactly half the time: `k * 2^(k - 1)` total ones.
2. From `2^k..n`, the highest bit contributes `n - 2^k + 1` ones.
3. Lower bits in that suffix repeat the pattern `0..n - 2^k`.

JAVA

```java
static long totalSetBitsThrough(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("n must be nonnegative");
    }
    if (n == 0) {
        return 0;
    }
    int highestBit = 31 - Integer.numberOfLeadingZeros(n);
    int power = 1 << highestBit;
    long fullBlock = highestBit == 0
            ? 0
            : (long) highestBit * (power >>> 1);
    long highBitSuffix = (long) n - power + 1;
    return fullBlock + highBitSuffix
            + totalSetBitsThrough(n - power);
}
```

Use `long` for the answer. The count can exceed `Integer.MAX_VALUE` even when `n` is an `int`.

The recursion removes the highest selected bit, so depth is at most 31 and time is `O(log n)` under a numeric-value model.

TRACE IT | Dry run for n = 13

TEXT

```
highest power = 8, k = 3
ones in 0..7 = 3 * 4 = 12
high bit in 8..13 = 6
remaining pattern = total bits in 0..5 = 7
total = 12 + 6 + 7 = 25
```

5.5 Complement only the significant bits

Java's `~value` flips all 32 bits. Many interview problems define the complement only from bit zero through the highest set bit.

For positive `value`, construct an all-one mask covering that width:

JAVA

```java
static int significantComplement(int value) {
    if (value < 0) {
        throw new IllegalArgumentException("nonnegative value required");
    }
    if (value == 0) {
        return 1;
    }
    int highest = Integer.highestOneBit(value);
    int mask = (highest << 1) - 1;
    return value ^ mask;
}
```

This implementation fails when `highest` is the sign bit, but the nonnegative contract limits the highest bit to position 30. For wider domains, use `long` and handle its positive limit deliberately.

Example: `5 = 101`, significant mask `111`, result `010 = 2`.

5.6 Minimum XOR pair after sorting

Recognition signal

Find the pair with minimum XOR among nonnegative values.

Baseline

Check every pair in `O(n^2)` time.

Sorted-adjacency property

After sorting nonnegative values, a minimum-XOR pair appears among adjacent values.

High-level proof: if `a < b < c`, the first high bit on which `a` and `c` differ separates the ordered range. At least one adjacent boundary inside that range differs no earlier and therefore cannot have a larger most-significant XOR bit than the wider pair. Repeatedly narrowing yields an adjacent candidate.

JAVA

```java
static int minimumXorPair(int[] input) {
    if (input == null || input.length < 2) {
        throw new IllegalArgumentException("at least two values required");
    }
    int[] values = input.clone();
    for (int value : values) {
        if (value < 0) {
            throw new IllegalArgumentException("nonnegative values required");
        }
    }
    Arrays.sort(values);
    int best = Integer.MAX_VALUE;
    for (int index = 1; index < values.length; index++) {
        best = Math.min(best, values[index - 1] ^ values[index]);
    }
    return best;
}
```

Time is `O(n log n)` and auxiliary space is `O(n)` here because input ownership is preserved by cloning. Sorting the input in place changes the space and mutation contract.

Negative values complicate signed order. Clarify the domain instead of applying the property without qualification.

5.7 Distinct bitwise OR values of all subarrays

Recognition signal

The problem asks for all distinct OR results across contiguous subarrays.

For every ending index, keep the OR values of subarrays ending there. Extending with a new value can only turn bits on, never off. Therefore the number of distinct OR states ending at one position is bounded by the word width plus a small constant: each genuinely new value must add at least one bit.

JAVA

```java
static int distinctSubarrayOrCount(int[] values) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
    Set<Integer> all = new HashSet<>();
    Set<Integer> previous = Set.of();
    for (int value : values) {
        Set<Integer> current = new HashSet<>();
        current.add(value);
        for (int prior : previous) {
            current.add(prior | value);
        }
        all.addAll(current);
        previous = current;
    }
    return all.size();
}
```

For fixed-width integers, the per-ending frontier has `O(w)` distinct values, giving `O(nw)` expected set operations and up to `O(nw)` distinct results overall. This is a good SDE-2 discussion: the visible nested work is controlled by a monotonic bit property, not by the number of earlier indexes.

5.8 Add two integers without + or -

This problem tests decomposition into sum-without-carry and carry.

TEXT

```text
partial sum without carry = a ^ b
carry positions           = (a & b) << 1
```

Repeat until no carry remains.

JAVA

```java
static int addWithBits(int first, int second) {
    while (second != 0) {
        int carry = (first & second) << 1;
        first ^= second;
        second = carry;
    }
    return first;
}
```

The result follows Java `int` wraparound semantics, just like ordinary `+`. The loop terminates within the fixed width because carries move left until they leave the 32-bit representation.

This is an interview exercise, not a recommendation to replace `+` in normal code. If overflow must be detected, use `Math.addExact` or a wider checked representation.

5.9 Bitwise AND and OR monotonicity

As a subarray expands:

- its AND can only lose one bits; and
- its OR can only gain one bits.

This limits the number of distinct states at a fixed endpoint and can support compressed-frontier algorithms. XOR has no corresponding monotonicity: adding another value may flip positions in either direction.

Recognition shortcut:

Operation across a growing range	State movement
AND	one bits only disappear
OR	one bits only appear
XOR	bits may flip either way

When a problem asks about distinct range AND or OR values, look for a bounded frontier. When it asks about XOR, look instead for cancellation, prefixes, tries, or linear algebra.

5.10 Interview technique: baseline to optimization

Use this answer structure:

1. **Contract:** "Inputs are nonnegative `int` values, and the objective uses signed Java order."
2. **Baseline:** "All pairs cost `O(n^2)` and constant extra space."
3. **Signal:** "XOR is decided lexicographically from high bits to low bits."
4. **Data structure or identity:** "A binary trie lets me prefer the opposite bit at each level."
5. **Invariant:** "At bit `b`, the chosen prefix is the best achievable XOR prefix among stored values."
6. **Complexity:** "`O(nw)` time and up to `O(nw)` nodes for width `w`."
7. **Boundaries:** "I need two values, and signed versus unsigned behavior changes bit 31."
8. **Trade-off:** "For small inputs, the quadratic version is simpler and may allocate less."

That sequence demonstrates senior judgment. Starting with a memorized trie template does not.

5.11 Common traps

- Greedily maximizing low XOR bits before high bits.
- Ignoring the sign bit while allowing negative values.
- Claiming trie space is `O(n)` without acknowledging the width and object overhead.
- Looping from `left` to `right` near `Integer.MAX_VALUE` and overflowing the loop variable.
- Returning an `int` for the total number of set bits through a large `n`.
- Using `~value` when the problem asks for significant-bit complement.
- Sorting a caller-owned array without stating the mutation.
- Claiming all nested loops are quadratic when a bit frontier has at most `w` states.
- Replacing normal arithmetic with bit arithmetic when readability is more important.

5.12 SDE-2 follow-up questions

1. How would a primitive-array trie reduce allocation compared with object nodes?
2. How would you delete values or support duplicate counts in a dynamic trie?
3. What does maximum XOR mean for signed results, and how would unsigned comparison change it?
4. Why does range AND keep only a common prefix?
5. Prove the block formula for total set bits through `n`.
6. Why does OR produce a bounded frontier but XOR does not?
7. When would sorting be preferable to a trie for pair-XOR problems?
8. What overflow semantics does the bit-addition method implement?

RECAP | Chapter summary

- A maximum-XOR trie chooses opposite bits from high to low under a defined ordering contract.
- Range AND is the common binary prefix of its endpoints.
- Total set-bit counts decompose around the highest power-of-two block.
- Significant complement needs a logical-width mask.
- Minimum XOR among nonnegative values can be found among sorted neighbors.
- Growing AND and OR ranges have monotonic, width-bounded state frontiers.
- Bit addition separates XOR sum from shifted carry but still wraps like Java `int` addition.
- SDE-2 answers connect the baseline, invariant, constraints, proof, cost, and engineering trade-off.

TRY IT | Readiness checkpoint

Choose any three patterns in this chapter. Derive them on a whiteboard without code, implement them from the derivation, and answer the signed-input, space-cost, and baseline-comparison follow-ups.

SDE-2 CORE CHAPTER

Chapter 6 - Java APIs, Production Choices, and Rapid Revision

Bits are a representation decision

An interview solution may fit in one `int`. A production design must also communicate names, ownership, concurrency, compatibility, and validation. This chapter helps you choose the simplest correct representation and defend that choice at SDE-2 depth.

Learning objectives

After this chapter, you should be able to:

- choose among `int`, `long`, `BitSet`, `EnumSet`, `BigInteger`, arrays, and sets;
- use Java's built-in bit APIs accurately;
- design a named and versioned packed-field API;
- reason about concurrent updates and serialization boundaries;
- test signed, width, and occurrence-count edge cases;
- identify common interview traps quickly; and
- use a concise decision guide during timed interviews.

6.1 Representation decision table

Representation	Best fit	Strengths	Main costs or risks
`int` mask	at most 32 fixed positions	primitive, compact, fast copying	sign/shift traps, unnamed positions
`long` mask	at most 64 fixed positions	larger primitive universe	`1L` and width-64 cases matter
`EnumSet<E>`	named enum flags inside Java	type safe, expressive, compact implementation	not a stable external binary schema
`BitSet`	dynamic dense nonnegative indexes	grows beyond 64, bulk operations	mutable, index mapping still needs meaning
`BigInteger`	immutable arbitrary-width bit pattern	immutable, signed numeric and bit APIs	allocations and signed semantics require care
`boolean[]`	bounded flags with direct indexing	obvious operations	more space, copying and equality policy
`HashSet<K>`	sparse or unbounded identifiers	expressive, supports arbitrary keys	allocation and expected hash costs

Choose based on semantics first. A primitive mask is not automatically the best design merely because it is compact.

6.2 Essential Integer and Long APIs

Question	Java method
How many one bits?	`Integer.bitCount(x)`
Where is the lowest one?	`Integer.lowestOneBit(x)`
Where is the highest one?	`Integer.highestOneBit(x)`
How many zeros before the highest one?	`Integer.numberOfLeadingZeros(x)`
How many zeros after the lowest one?	`Integer.numberOfTrailingZeros(x)`
Reverse all positions?	`Integer.reverse(x)`
Reverse byte order?	`Integer.reverseBytes(x)`
Rotate without losing bits?	`Integer.rotateLeft(x, d)` / `rotateRight`
Compare unsigned?	`Integer.compareUnsigned(a, b)`
Widen unsigned `int`?	`Integer.toUnsignedLong(x)`
Show unsigned decimal?	`Integer.toUnsignedString(x)`

`reverse` and `reverseBytes` are different:

```text
reverse:      bit 0 exchanges with bit 31, bit 1 with bit 30, ...
reverseBytes: byte 0 exchanges with byte 3, preserving bit order inside a byte
```

Prefer named library methods in production code. In an interview, be ready to derive the underlying loop when that is the skill being tested.

6.3 BitSet basics

`BitSet` represents a growable set of nonnegative integer positions.

```java
BitSet available = new BitSet();
available.set(2);
available.set(5);
available.flip(5);
boolean hasTwo = available.get(2);
int selected = available.cardinality();
int first = available.nextSetBit(0);
```

Bulk operations mutate the receiver:

```java
BitSet intersection = (BitSet) firstSet.clone();
intersection.and(secondSet);

BitSet union = (BitSet) firstSet.clone();
union.or(secondSet);
```

Do not mutate an input accidentally. Clone when ownership requires preservation.

Important behaviors:

- logical length is highest selected index plus one, not allocated capacity;
- `size()` reports internal storage capacity and is not the number of selected bits;
- `cardinality()` counts selected positions;
- negative indexes throw `IndexOutOfBoundsException`; and
- equality compares logical selected bits, not spare capacity.

For dense sets much larger than 64, `BitSet` can be clearer and more efficient than a boxed `HashSet<Integer>`.

6.4 EnumSet for named flags

JAVA

```java
enum Permission {
    READ,
    WRITE,
    EXPORT,
    ADMIN
}

EnumSet<Permission> permissions = EnumSet.of(
        Permission.READ, Permission.EXPORT);

if (permissions.contains(Permission.EXPORT)) {
    // named capability
}
```

`EnumSet` communicates intent and rejects flags from the wrong enum type. It is usually better than scattering raw bit positions through application code.

Do not persist `EnumSet` by assuming its internal bit mapping is a permanent protocol. Enum ordering can change, and implementation details are not an external schema. Define explicit wire codes when data crosses process or storage boundaries.

6.5 BigInteger for arbitrary-width immutable bits

Useful methods include:

JAVA

```java
BigInteger value = BigInteger.ZERO.setBit(100);
boolean selected = value.testBit(100);
BigInteger cleared = value.clearBit(100);
BigInteger toggled = value.flipBit(7);
int count = value.bitCount();
```

`BigInteger` is immutable, so each update returns a new value. It also represents signed mathematical integers using two's-complement-like infinite sign extension for bit operations. For a finite unsigned protocol, define and enforce the logical width rather than assuming `not()` creates a bounded complement.

6.6 A named 64-bit flag API

Avoid magic positions at call sites:

JAVA

```
enum AccountFlag {
    VERIFIED(0),
    PREMIUM(1),
    EXPORT_ALLOWED(2),
    REVIEW_REQUIRED(3);

    private final long mask;

    AccountFlag(int index) {
        this.mask = 1L << index;
    }

    long mask() {
        return mask;
    }
}

static boolean hasFlag(long flags, AccountFlag flag) {
    Objects.requireNonNull(flag, "flag");
    return (flags & flag.mask()) != 0;
}

static long withFlag(long flags, AccountFlag flag) {
    Objects.requireNonNull(flag, "flag");
    return flags | flag.mask();
}
```

The API gives the bit a domain name. A production design should also specify:

- logical width;
- reserved positions;
- meaning of zero;
- unknown-bit policy;
- persistence and network byte order;
- versioning and migration policy; and
- whether callers may combine arbitrary raw masks.

6.7 Packed fields require validation

Suppose a 64-bit word contains a 3-bit status field beginning at offset 8. Values from 0 through 7 fit. A value of 8 must be rejected rather than truncated to zero.

JAVA

```java
static long packUnsignedField(
        long word, int offset, int width, long fieldValue) {
    if (offset < 0 || width < 1 || width > Long.SIZE
            || offset > Long.SIZE - width) {
        throw new IllegalArgumentException("invalid field bounds");
    }
    long lowMask = width == Long.SIZE ? -1L : (1L << width) - 1;
    if ((fieldValue & ~lowMask) != 0) {
        throw new IllegalArgumentException("field value does not fit");
    }
    if (width == Long.SIZE) {
        return fieldValue;
    }
    long shiftedMask = lowMask << offset;
    return (word & ~shiftedMask) | (fieldValue << offset);
}
```

If negative field values are allowed, define signed-field encoding separately. The unsigned validation above deliberately rejects them.

6.8 Concurrency: primitive does not mean compound update is atomic

Two threads execute:

JAVA

```java
flags = flags | mask;
```

Each operation reads the old value, computes a new value, and writes it. Two updates can race and one may overwrite the other even though each individual primitive read or write has defined atomicity properties.

Use a lock or an atomic read-modify-write operation:

JAVA

```java
AtomicLong flags = new AtomicLong();

static void enable(AtomicLong flags, long mask) {
    flags.getAndUpdate(current -> current | mask);
}
```

An SDE-2 answer should distinguish representation from synchronization. Compact state does not automatically create a correct concurrent API.

6.9 Serialization and endianness

Bit numbering inside a numeric value and byte order in a serialized stream are related but distinct choices.

For a `ByteBuffer`:

JAVA

```
ByteBuffer buffer = ByteBuffer.allocate(Long.BYTES)
        .order(ByteOrder.BIG_ENDIAN);
buffer.putLong(flags);
```

The protocol must specify byte order. It must also specify which numeric bit position represents each field. Do not use `Integer.reverseBytes` or `Long.reverseBytes` unless converting an explicitly defined byte order.

6.10 Trust-boundary validation

When flags come from a client or stored record:

JAVA

```
long knownMask = AccountFlag.VERIFIED.mask()
        | AccountFlag.PREMIUM.mask()
        | AccountFlag.EXPORT_ALLOWED.mask()
        | AccountFlag.REVIEW_REQUIRED.mask();
long unknown = incomingFlags & ~knownMask;
```

Choose an explicit policy:

- reject unknown bits;
- preserve unknown optional bits when forwarding;
- clear unknown bits; or
- accept them only under a newer schema version.

Never grant authorization merely because an unrecognized bit is set. Translate raw values into named, validated capabilities at the boundary.

6.11 Testing matrix for bit code

Test more than ordinary positive examples.

Category	Representative cases
zero	no selected bits, empty subset, zero prefix
one-bit values	bit 0, middle bit, highest legal bit
all ones	`-1`, logical-width all-one mask
sign boundary	`Integer.MIN_VALUE`, `Long.MIN_VALUE`
range boundaries	index `0`, `31`, `63`, and invalid neighbors
occurrence contracts	negative single, malformed duplicates, equal exceptions
output size	`n = 0`, small maximum, rejected large subset input
fields	width 1, full width, value just too large
prefix queries	one element, whole array, invalid endpoints
signed policy	compare same bits using signed and unsigned APIs

Property checks are especially valuable:

TEXT

```
clear(set(x, i), i) has bit i clear
toggle(toggle(x, i), i) == x
bitCount(x) == bitCount(x & (x - 1)) + 1 for x != 0
rangeXor(prefix, i, i) == values[i]
gray(i) and gray(i + 1) differ in one bit
```

6.12 Mandatory Java traps

1. `1 << 40` is an `int` shift with effective distance 8.
2. `1L << 64` has effective distance zero.
3. `~(byte) 0` is a 32-bit `int` result.
4. `(value & mask) == 1` only works for mask one.
5. `x & (x - 1) == 0` needs parentheses and a positive contract.
6. `>>` and `/ 2` differ for negative odd values.
7. `>>` sign-extends while `>>>` zero-fills.
8. `~mask` flips positions outside a logical subset universe.
9. `x & -x` can produce a negative sign-bit mask.
10. `Integer.numberOfTrailingZeros(0)` returns 32.
11. XOR cancellation does not validate duplicate counts.
12. XOR result ordering may require unsigned comparison.
13. `1 << n` does not make large subset enumeration safe.
14. output-sized subset storage is not `O(1)`.
15. a read-modify-write flag update can lose concurrent changes.
16. `BitSet.size()` is not selected-bit count.
17. `BitSet.and` and `or` mutate the receiver.
18. `BigInteger.not()` is not a finite-width complement.
19. `final` prevents reference reassignment; it does not make a mutable `BitSet` immutable.
20. byte order must be explicit when packed values cross systems.

CHOOSE IT | 6.13 The interview decision guide

If the prompt says...	Ask...	First candidate
flags, permissions, chosen items	Is the universe small, fixed, and dense?	primitive mask or `EnumSet`
one bit differs or flips	Is it a comparison of two patterns?	XOR then `bitCount`
paired duplicates	What is the exact multiplicity contract?	XOR cancellation
two exceptional values	Are they guaranteed distinct?	XOR partition by low bit
range XOR queries	Is the array immutable?	prefix XOR
target XOR subarrays	Do I need counts of prior prefixes?	prefix-frequency map
all subsets	Is `2^n` feasible and output required?	mask enumeration
all submasks	How many bits are selected?	`(sub - 1) & mask`
maximum XOR	What signed/unsigned order is intended?	high-to-low trie
range AND	Which high prefix stays constant?	shift to common prefix
many range OR/AND states	Does monotonicity bound distinct states?	compressed frontier

6.14 A five-minute rapid revision sheet

TEXT

```
test bit i:       (x & (1L << i)) != 0
set bit i:        x | (1L << i)
clear bit i:      x & ~(1L << i)
toggle bit i:     x ^ (1L << i)
lowest one:       x & -x
remove lowest:    x & (x - 1)
positive power 2: x > 0 && (x & (x - 1)) == 0
Hamming distance: bitCount(a ^ b)
low width mask:   (1L << width) - 1, except width 64
range XOR:        prefix[r + 1] ^ prefix[l]
XOR 0..n:         n, 1, n + 1, 0 by n mod 4
next submask:     (sub - 1) & mask
Gray code:        i ^ (i >>> 1)
```

For every line, remember the width and contract. A sheet is useful for retrieval only after you can derive it.

6.15 How to communicate a bit solution

Use this compact script:

1. "I will treat each value as a 32-bit Java pattern. Negative values are allowed/not allowed."
2. "The input promise is ..."
3. "The baseline is ..."
4. "The useful identity or monotonic property is ... because ..."
5. "My loop invariant is ..."
6. "Time is ... and auxiliary/output space is ..."
7. "I will test zero, the highest position, negative or unsigned behavior, and malformed contract cases."
8. "In production I would use the named Java API or representation ..."

That is more convincing than saying "this is a known bit trick."

RECAP | Chapter summary

- Primitive masks suit small fixed universes; `EnumSet`, `BitSet`, `BigInteger`, arrays, and sets cover different semantics.
- Java provides clear, optimized bit operations that should be preferred when allowed.
- Packed state is a schema requiring names, bounds, versioning, byte order, and unknown-bit policy.
- Compound concurrent updates need synchronization or atomic read-modify-write operations.
- Tests must cover zero, sign bits, full width, invalid indexes, occurrence contracts, and output limits.
- The best interview technique begins with the contract and derivation, then uses a concise implementation.

TRY IT | Final learning checkpoint before practice

You are ready for the practice lab when you can use the decision guide without confusing signed and unsigned behavior, explain at least ten traps, and choose a Java representation based on semantics rather than cleverness.

SDE-2 CORE CHAPTER

Chapter 7 - Bit Manipulation Practice Lab

How to use this lab

Keep a separate answer sheet. For every code question:

1. state the width and signed/unsigned interpretation;
2. draw only the relevant bit positions;
3. predict before running;
4. record the invariant or identity; and
5. state time, auxiliary space, and any output space.

Difficulty labels mean:

- **Foundation:** representation and operator fluency;
- **Interview Core:** reusable coding-interview patterns; and
- **SDE-2 Follow-up:** proof, boundary, API, or engineering depth.

Solutions are in the next chapter. Do not read them until you have written and tested an answer.

Part A - Knowledge checks

Foundation

K01. What decimal contribution does bit position 6 make when selected?

K02. Why does bit numbering begin at zero in the usual mask formula?

K03. Explain the difference between a bit pattern and its signed interpretation.

K04. What does AND do at one position? Give one use case.

K05. What does OR do at one position? Give one use case.

K06. What does XOR do at one position? Give one use case.

K07. Why is `~5` equal to `-6` rather than a small positive value?

K08. Compare `>>` and `>>>`.

K09. Why can `x << 1` differ from `x * 2` as a problem-solving replacement even though Java wraparound results match for ordinary `int` arithmetic?

K10. Why does `byte` bitwise arithmetic normally produce an `int`?

Interview Core

K11. Derive test, set, clear, and toggle for bit `i`.

K12. Why must a bit test compare with nonzero instead of one?

K13. Prove that `x & (x - 1)` removes the lowest set bit.

K14. Prove that `x & -x` isolates the lowest set bit.

K15. Why does a power-of-two test require `x > 0`?

K16. What does `Integer.bitCount(-1)` return and why?

K17. Why does a half-open prefix XOR make inclusive queries convenient?

K18. State the exact input promise for one exceptional value among pairs.

K19. Why does one distinguishing bit separate two exceptional values?

K20. Why must the sign bit be included when reconstructing one value among triples?

K21. What is the difference between index subsets and unique value subsets?

K22. Why is materializing every subset not constant auxiliary/output space?

K23. Derive the number of submasks of a mask with `k` set bits.

K24. Explain the three roles per position behind the `O(3^n)` mask-submask bound.

SDE-2 Follow-up

K25. When should an API use `EnumSet` instead of a raw primitive mask?

K26. Why can `flags = flags | mask` lose a concurrent update?

K27. What must be versioned when a packed word is persisted or sent over a network?

K28. Why is maximum XOR greedy from high bits to low bits?

K29. Why does range AND retain only the common high prefix?

K30. Explain why a growing OR frontier is width-bounded while a growing XOR frontier is not monotonic.

Part B - Predict the output

Assume Java 21. Do not run the snippets until after predicting.

Foundation

O01.

JAVA

```java
System.out.println(10 & 6);
System.out.println(10 | 6);
System.out.println(10 ^ 6);
```

O02.

JAVA

```java
System.out.println(~0);
System.out.println(~7);
```

O03.

JAVA

```java
System.out.println(1 << 5);
System.out.println(1 << 32);
System.out.println(1L << 32);
```

O04.

JAVA

```java
System.out.println(-1 >> 1);
System.out.println(-1 >>> 1);
```

O05.

JAVA

```java
byte value = 5;
System.out.println(~value);
System.out.println((~value) & 0xFF);
```

O06.

JAVA

```java
int value = 0b1010;
int mask = 1 << 3;
System.out.println((value & mask) != 0);
System.out.println(value ^ mask);
```

Interview Core

O07.

JAVA

```java
int x = 0b10110000;
System.out.println(Integer.toBinaryString(x & (x - 1)));
System.out.println(Integer.toBinaryString(x & -x));
```

O08.

JAVA

```java
System.out.println(Integer.bitCount(0));
System.out.println(Integer.bitCount(-1));
System.out.println(Integer.numberOfTrailingZeros(0));
```

O09.

JAVA

```java
int x = 8;
System.out.println(x > 0 && (x & (x - 1)) == 0);
x = 0;
System.out.println(x > 0 && (x & (x - 1)) == 0);
```

O10.

JAVA

```java
int result = 0;
for (int value : new int[] {4, 1, 4, 9, 1}) {
    result ^= value;
}
System.out.println(result);
```

O11.

JAVA

```java
int prefix = 0;
for (int value : new int[] {3, 3, 5, 5, 7}) {
    prefix ^= value;
}
System.out.println(prefix);
```

O12.

JAVA

```java
for (int n = 0; n < 8; n++) {
    System.out.print(xorZeroThrough(n) + " ");
}
```

Use the four-case method from Chapter 3.

O13.

JAVA

```java
int mask = 0b10110;
int count = 0;
for (int sub = mask; sub != 0; sub = (sub - 1) & mask) {
    count++;
}
System.out.println(count);
```

O14.

JAVA

```java
for (int i = 0; i < 4; i++) {
    System.out.print((i ^ (i >>> 1)) + " ");
}
```

SDE-2 Follow-up

O15.

JAVA

```java
System.out.println(Integer.compare(-1, 1));
System.out.println(Integer.compareUnsigned(-1, 1));
System.out.println(Integer.toUnsignedLong(-1));
```

O16.

JAVA

```java
BitSet bits = new BitSet();
bits.set(2);
bits.set(100);
System.out.println(bits.length());
System.out.println(bits.cardinality());
```

O17.

JAVA

```java
int first = 3;
int second = 5;
while (second != 0) {
    int carry = (first & second) << 1;
    first ^= second;
    second = carry;
}
System.out.println(first);
```

O18.

JAVA

```
System.out.println(Integer.highestOneBit(13));
System.out.println(31 - Integer.numberOfLeadingZeros(13));
```

O19.

JAVA

```
int left = 26;
int right = 30;
int shifts = 0;
while (left != right) {
    left >>>= 1;
    right >>>= 1;
    shifts++;
}
System.out.println(left << shifts);
```

O20.

JAVA

```
int value = Integer.MIN_VALUE;
System.out.println(value & -value);
System.out.println((value & (value - 1)) == 0);
```

Part C - Debug the code

For each task, identify the defect, give a failing input, and repair the implementation.

Foundation

D01. Check bit

JAVA

```
static boolean isSet(int value, int index) {
    return (value & (1 << index)) == 1;
}
```

D02. Long mask

JAVA

```
static long set(long value, int index) {
    return value | (1 << index);
}
```

D03. Index validation

```java
static int mask(int index) {
    return 1 << index;
}
```

D04. Clear bit

```java
static int clear(int value, int index) {
    return value ^ (1 << index);
}
```

D05. Unsigned byte inversion

```java
static int invertByte(byte value) {
    return ~value;
}
```

Interview Core

D06. Power of two

```java
static boolean isPowerOfTwo(int value) {
    return (value & (value - 1)) == 0;
}
```

D07. Count bits of a negative value

```java
static int countBits(int value) {
    int count = 0;
    while (value > 0) {
        count += value & 1;
        value >>= 1;
    }
    return count;
}
```

D08. Missing value

JAVA

```java
static int missing(int[] values) {
    int answer = 0;
    for (int index = 0; index < values.length; index++) {
        answer ^= index ^ values[index];
    }
    return answer;
}
```

D09. Prefix range

JAVA

```java
static int rangeXor(int[] prefix, int left, int right) {
    return prefix[right] ^ prefix[left];
}
```

D10. Two exceptional values

JAVA

```java
int distinguishing = Math.abs(combined & -combined);
```

D11. Triple occurrence reconstruction

JAVA

```java
for (int bit = 0; bit < 31; bit++) {
    // count and reconstruct
}
```

D12. Subset bound

JAVA

```java
long total = 1 << values.length;
```

D13. Logical complement

JAVA

```java
static int complement(int value) {
    return ~value;
}
```

D14. Submask loop

JAVA

```java
for (int sub = mask; ; sub = (sub - 1) & mask) {
    process(sub);
}
```

D15. Target XOR count

JAVA

```java
Map<Integer, Integer> frequency = new HashMap<>();
int prefix = 0;
long count = 0;
for (int value : values) {
    prefix ^= value;
    count += frequency.getOrDefault(prefix ^ target, 0);
    frequency.merge(prefix, 1, Integer::sum);
}
```

SDE-2 Follow-up

D16. Packed field

JAVA

```java
long updated = word | (fieldValue << offset);
```

D17. Concurrent flags

JAVA

```java
volatile long flags;

void enable(long mask) {
    flags = flags | mask;
}
```

D18. `BitSet` intersection

JAVA

```java
static BitSet intersection(BitSet first, BitSet second) {
    first.and(second);
    return first;
}
```

D19. Total set-bit count type

JAVA

```java
static int totalSetBitsThrough(int n) {
    // mathematically correct recurrence returning int
}
```

D20. Maximum XOR with negative inputs

JAVA

```java
// Trie scans bits 30..0 and returns Math.max of int XOR results.
```

Part D - Small coding tasks

Foundation

C01. Implement a padded 32-bit binary formatter without using `String.format`.

C02. Implement validated `long` helpers for test, set, clear, and toggle.

C03. Return a mask containing the lowest `width` bits for every width from 0 through 64.

C04. Extract and replace an unsigned field in a `long`.

C05. Count set bits with both a position scan and Kernighan's method. Compare iteration counts.

C06. Implement positive power-of-two and power-of-four checks.

C07. Return the Hamming distance between two `long` values.

C08. Reverse all 32 bits of an `int` without `Integer.reverse`.

Interview Core

C09. Find one exceptional value among pairs.

C10. Find the missing value from distinct `0..n` input.

C11. Find two exceptional values among pairs and return them in sorted order.

C12. Find one value among triples using per-bit counts; support a negative answer.

C13. Build a prefix-XOR array and answer validated inclusive queries.

C14. Compute XOR for an inclusive nonnegative integer range in constant time.

C15. Count subarrays whose XOR equals a target; return `long`.

C16. Generate index subsets for at most 20 values and state output space.

C17. Enumerate every submask, including zero exactly once.

C18. Generate the first `2^n` Gray codes and assert consecutive Hamming distance one.

SDE-2 Follow-up

C19. Implement maximum XOR pair for nonnegative values using both `O(n^2)` and a trie; differential-test them.

C20. Implement range bitwise AND using both common-prefix shifts and lowest-bit clearing.

C21. Count all set bits from one through `n` using `long` output.

C22. Find the minimum XOR pair after cloning and sorting the input.

C23. Count distinct subarray OR results using a frontier set.

C24. Design a versioned permission value with named flags, unknown-bit validation, and atomic updates.

Part E - Interview follow-up chains

F01. You solved set-bit count. How do fixed-width and arbitrary-precision complexity models differ?

F02. You used `1L << index`. What should happen for index 64 or -1?

F03. You solved power of two. How do zero and `Integer.MIN_VALUE` expose missing contracts?

F04. You found a single value with XOR. How would you validate the occurrence promise?

F05. You found two singles. What if their distinguishing bit is the sign bit?

F06. You used prefix XOR. How would point updates change the data-structure choice?

F07. You counted target-XOR subarrays. What is the maximum answer and why is `long` useful?

F08. You generated subsets. At what `n` does output become the limiting factor before mask width?

F09. You used bitmask DP. What information is absent from the mask, and how do you know it is safe to omit?

F10. You built a trie. Compare object nodes, primitive arrays, and a quadratic baseline.

F11. You solved range AND. What property fails if the operator is XOR?

F12. You used `BitSet`. Discuss mutability and method side effects.

F13. You used a primitive permission mask. How do schema evolution and unknown bits work?

F14. Two threads set different bits. Make the operation linearizable.

F15. A network protocol provides eight bytes. Separate byte-order handling from numeric bit numbering.

Part F - Cumulative assessments

Assessment 1 - Foundation reconstruction

Time box: 35 minutes.

1. Draw `-13` as a 32-bit pattern using two's-complement reasoning.
2. Predict `-13 >> 2` and `-13 >>> 2`.
3. Implement all four one-bit operations for `long` with index validation.
4. Derive low-bit isolation and removal.
5. Explain promotion for `byte flags = 5; int x = ~flags;`.

Pass standard: four of five correct, with no confusion between `>>` and `>>>` or between `int` and `long` shift width.

Assessment 2 - Pattern selection

Time box: 55 minutes.

Solve and explain:

1. one value among pairs;
2. two values among pairs;
3. inclusive range XOR queries;
4. target-XOR subarray count; and
5. every submask of a given mask.

Pass standard: correct contracts, invariants, boundaries, and complexity for at least four problems. No unexplained memorized expression.

Assessment 3 - SDE-2 design and optimization

Time box: 70 minutes.

1. Give baseline and trie solutions for maximum XOR.
2. Derive range AND from common-prefix behavior.
3. Design a versioned 64-bit permissions API with atomic updates.
4. Compare `long`, `EnumSet`, `BitSet`, and `HashSet` for three workloads.
5. Review a colleague's subset generator for overflow, duplicate semantics, and output-space claims.

Pass standard: implementation correctness plus explicit trade-offs, mutation/ownership, signedness, and validation.

Final readiness assessment

You are ready to continue to Loop Mastery and Arrays when all statements are true:

- [ ] I can derive operator results from truth tables.
- [ ] I validate Java shift indexes and choose `1` versus `1L` correctly.
- [ ] I can derive every one-bit mask operation.
- [ ] I can prove low-bit isolation and removal.
- [ ] I can solve the three main occurrence-count XOR families.
- [ ] I can build prefix XOR and a prefix-frequency solution.
- [ ] I report subset and submask complexity honestly.
- [ ] I can explain maximum XOR, range AND, and total set-bit patterns.
- [ ] I can identify at least fifteen Java bit traps.
- [ ] I choose representations based on semantics and constraints.
- [ ] I can discuss packed-schema versioning and atomic updates.
- [ ] I passed all three cumulative assessments without using the solution chapter.

Recommended threshold: at least 80 percent on knowledge and output questions, working implementations for C01-C18, and defensible solutions for at least four of C19-C24. Record every missed contract or width error in an interview error log and repeat the matching section after 48 hours.

SDE-2 CORE CHAPTER

Chapter 8 - Practice Solutions and Reasoning

How to review

Mark an answer correct only when its reasoning and contract are correct. A lucky output prediction or copied expression is not mastery. For coding tasks, compare invariants and edge cases before comparing syntax.

Part A - Knowledge-check solutions

K01. Bit position 6 contributes `2^6 = 64` when selected.

K02. Position zero has weight `2^0 = 1`. Shifting the value one left by position `i` therefore produces the mask for weight `2^i`.

K03. A pattern is the fixed sequence of zeros and ones. Signed and unsigned rules assign different numeric meanings to the same pattern. For example, all 32 one bits represent `-1` as a signed `int` and `4_294_967_295` as an unsigned magnitude.

K04. AND produces one only when both input bits are one. It can test a selected bit, retain a field, or clear positions by ANDing with an inverted mask.

K05. OR produces one when either bit is one. It sets selected flags or combines disjoint flag groups.

K06. XOR produces one when bits differ. It toggles selected positions, measures differences, and cancels equal fixed-width values under a matching occurrence contract.

K07. Java inverts all 32 `int` positions. In two's complement, `~x == -x - 1`, so `~5 == -6`.

K08. Both shift right. `>>` copies the sign bit into high positions; `>>>` fills them with zero. They match for nonnegative inputs but differ for negative values.

K09. Both ordinary `int` operations wrap, but a shift discards bits and uses a masked distance. More importantly, replacing arithmetic with a shift can violate a problem contract involving checked overflow, negative rounding, a variable distance, or clarity. Use a shift when bit structure is the intent.

K10. Java applies unary or binary numeric promotion to `byte`, `short`, and `char`, normally producing an `int` before the bitwise operation.

K11. With `mask = 1L << i`: test with `(x & mask) != 0`; set with `x | mask`; clear with `x & ~mask`; toggle with `x ^ mask`. Each follows directly from the one-bit truth table.

K12. The retained value equals the mask, not necessarily one. If bit 5 is selected, the nonzero result is 32.

K13. Subtracting one clears the lowest one and turns every lower zero into one. AND preserves the unchanged high prefix and clears the changed suffix, removing exactly that one.

K14. `-x` equals `~x + 1`. At the lowest one of `x`, the carry makes the corresponding bit in `-x` one; lower bits are zero in `x`, and higher common ones do not survive the construction. AND leaves only the lowest one.

K15. Zero satisfies the raw identity, and `Integer.MIN_VALUE` has one selected representation bit. The usual problem defines positive mathematical powers of two, so `x > 0` is required.

K16. It returns 32 because `-1` is all ones in the 32-bit two's-complement representation.

K17. `prefix[i]` represents exactly the first `i` elements. The inclusive range `[l, r]` is therefore `prefix[r + 1] ^ prefix[l]`, avoiding a special case at `l = 0`.

K18. The input is nonempty, exactly one value occurs once, and every other value occurs exactly twice. If validation is not implemented, the method relies on that promise.

K19. If `a != b`, then `a ^ b` has at least one one bit. At that position one exceptional value has zero and the other has one. Equal pairs have equal bits and enter the same group, where they cancel.

K20. A negative exceptional value is identified partly by bit 31. Omitting it reconstructs a nonnegative value with the wrong representation.

K21. Two masks may select different indexes containing equal values. They are distinct index subsets but can produce identical value lists.

K22. There are `2^n` subset objects and `Theta(n * 2^n)` selected-element occurrences across them. Working variables may be small, but the required result is not.

K23. Each of the `k` selected positions may be absent or present in a submask, giving `2^k` choices including zero.

K24. Each universe position is outside the mask, inside the mask but outside the submask, or inside both. Three independent choices across `n` positions give `3^n` relationships.

K25. Prefer `EnumSet` when flags are named Java enum values inside one process and a fixed wire representation is unnecessary. It provides type safety and intent while remaining compact.

K26. The expression is read-compute-write. Two threads can read the same old value, set different bits, then have the later write erase the earlier change.

K27. Version logical width, position meanings, reserved and required bits, default values, unknown-bit policy, signedness where relevant, and serialized byte order.

K28. In binary numeric ordering, a one at a higher position outweighs every combination of lower positions. A trie therefore chooses the opposite branch at the highest available position before optimizing lower bits.

K29. When the range crosses a bit boundary, that bit is zero for at least one value and is cleared by the total AND. Only the high prefix shared by both endpoints never changes across the range.

K30. OR can only add one bits as a range grows, so every new distinct state must add a position and there are at most `w` such changes. XOR can turn a bit on or off repeatedly and has no monotonic frontier.

Part B - Output solutions

O01. `2`, `14`, `12`. In four bits: `1010 & 0110 = 0010`, OR is `1110`, and XOR is `1100`.

O02. `-1`, `-8`. The identity is `~x = -x - 1`.

O03. `32`, `1`, `4294967296`. The `int` shift distance 32 becomes zero; the `long` shift retains distance 32.

O04. `-1`, `2147483647`. Arithmetic shift repeats one; logical shift inserts zero.

O05. `-6`, `250`. The byte promotes to `int`; `& 0xFF` retains only the low eight inverted positions.

O06. `true`, `2`. Bit 3 is selected in `1010`; toggling it produces `0010`.

O07. `10100000`, `10000`. The first removes the lowest one; the second isolates it. `toBinaryString` omits leading zeros.

O08. `0`, `32`, `32`. The last value is a zero-input sentinel, not a valid selected position.

O09. `true`, `false`. The positive condition rejects zero.

O10. `9`. The two fours and two ones cancel.

O11. `7`. Equal pairs cancel regardless of order.

O12. `0 1 3 0 4 1 7 0 `, including the trailing space printed by the loop.

O13. `7`. The mask has three ones, so it has `2^3 - 1` nonzero submasks.

O14. `0 1 3 2 `. These are the first four Gray codes.

O15. A negative number, a positive number, then `4294967295`. Exact comparison returns are specified only by sign, not necessarily `-1` and `1`; current methods return `-1` and `1`, but tests should rely on sign. Unsigned `-1` is the largest 32-bit magnitude.

O16. `101`, `2`. Logical length is highest selected index plus one; cardinality counts selected positions.

O17. `8`. XOR holds sum without carry and shifted AND holds the carry until none remains.

O18. `8`, `3`. The highest selected position of 13 is index 3 and its mask is 8.

O19. `24`. The common prefix of 26 and 30 is restored with trailing zeros.

O20. Both lines print `-2147483648` and `true`, respectively. The sign-bit mask is negative, and the raw one-bit identity holds. This is why positive power-of-two detection requires `value > 0`.

Part C - Debugging solutions

D01. Comparing with one fails for every selected bit except bit zero. Use `(value & (1 << index)) != 0` after validating `0..31`.

D02. The mask is calculated as an `int`; index 40 becomes effective distance 8. Use `1L << index` and validate `0..63`.

D03. Java silently masks invalid shift distances. Reject indexes outside `0..31` before shifting.

D04. XOR toggles and will set an already-clear bit. Use `value & ~(1 << index)`.

D05. `~value` is a promoted 32-bit result. For unsigned inverted byte content, return `(~value) & 0xFF`.

D06. Zero and `Integer.MIN_VALUE` pass the raw one-bit expression. Use `value > 0 && (value & (value - 1)) == 0`.

D07. `while (value > 0)` skips every negative input. Scan exactly 32 positions with `>>>`, or use `while (value != 0) value &= value - 1`.

D08. The expected range includes `n = values.length`, which is never XORed. Initialize `answer = values.length` or XOR it separately.

D09. For a half-open prefix, the inclusive range is `prefix[right + 1] ^ prefix[left]`. Add bounds validation.

D10. The isolated sign-bit mask is `Integer.MIN_VALUE`; `Math.abs` cannot make it positive. Use `combined & -combined` directly as a mask.

D11. Bit 31 is missing, so a negative exceptional result cannot be reconstructed. Loop through `bit < Integer.SIZE`.

D12. `1 << length` is calculated as `int` before widening. Use `1L << length` and reject infeasible or width-invalid lengths.

D13. ~ flips all 32 positions. For a nonnegative significant-width complement, XOR with an all-one mask through the highest selected bit; define zero as a special case.

D14. After `sub` reaches zero, the update returns `mask`, creating a cycle. Loop while `sub != 0` and process zero after the loop, or break immediately after processing zero.

D15. The map is missing the empty prefix. Add `frequency.put(0, 1)` before the loop so subarrays beginning at index zero are counted.

D16. OR does not clear the old field and does not validate width. Build the range mask, verify the new value fits, clear old positions, then OR the shifted value.

D17. `volatile` provides visibility and ordering for the variable but not atomicity for the read-modify-write workflow. Use a lock or `AtomicLong.getAndUpdate(current -> current | mask)`.

D18. `BitSet.and` mutates `first`. If inputs must be preserved, clone first, mutate the clone, and return it.

D19. The answer may exceed `int`. Return `long` and widen intermediate multiplication before it overflows.

D20. Scanning only bits 30..0 ignores the sign bit, and `Math.max` uses signed order. Either reject negatives, or scan all 32 bits and define comparison using `Integer.compareUnsigned` or the intended signed objective.

Part D - Coding-task solution guidance

C01 - Padded formatter

JAVA

```java
static String bits32(int value) {
    StringBuilder result = new StringBuilder(Integer.SIZE);
    for (int bit = Integer.SIZE - 1; bit >= 0; bit--) {
        result.append((value >>> bit) & 1);
    }
    return result.toString();
}
```

Exactly 32 iterations preserve leading zeros and negative representations.

C02 - Validated operations

Centralize validation in a helper returning `1L << index`, then use AND, OR, AND-NOT, and XOR. Test indexes 0 and 63 plus -1 and 64.

C03 - Low-width mask

Return zero for width zero, `-1L` for width 64, and `(1L << width) - 1` otherwise. Reject widths outside `0..64`.

C04 - Field extraction and replacement

Validate `offset >= 0`, `width >= 1`, and `offset <= 64 - width`. Extract with `>>> offset` and a low mask. Replace by validating `fieldValue`, clearing the shifted range, and ORing the shifted value.

C05 - Two bit counts

The scan always uses 32 iterations. Kernighan uses `p` iterations for `p` one bits. Both return the same count for negative values when the scan uses `>>>` or direct mask tests.

C06 - Powers

Power of two: positive plus one-bit test. Power of four: power of two plus intersection with `0x5555_5555`.

C07 - Long Hamming distance

```java
return Long.bitCount(first ^ second);
```

XOR selects differing positions across the full 64-bit representation.

C08 - Reverse bits

Repeat 32 times: shift result left, insert `value & 1`, and consume `value >>>= 1`. Compare randomly with `Integer.reverse`.

C09 - One among pairs

XOR reduction is sufficient under the promise. State `O(n)` time, `O(1)` space, and no occurrence validation.

C10 - Missing 0 through n

Initialize answer to `values.length`, then XOR each index and observed value. Test missing zero and missing `n`.

C11 - Two singles

XOR all, isolate `combined & -combined`, partition, XOR within groups, then sort the two results. Do not use `Math.abs` on the mask.

C12 - One among triples

For all 32 bits, sum `(value >>> bit) & 1`, reduce modulo three, and OR the remaining bit. Bit 31 support is mandatory.

C13 - Prefix XOR

Build length `n + 1` with `prefix[i + 1] = prefix[i] ^ values[i]`. Query inclusive range with `prefix[r + 1] ^ prefix[l]` after validation.

C14 - Integer range XOR

Use the `n & 3` cycle for `0..n`, then XOR the two prefixes. Handle `left == 0` without computing `left - 1`.

C15 - Target subarrays

Start prefix frequency with zero once. At each element, add the frequency of `prefix ^ target` before recording the current prefix. Return `long`.

C16 - Subsets

Use masks `0..(1 << n) - 1`, map positions to array indexes, and materialize immutable copies only if required. Report `Theta(n * 2^n)` output size.

C17 - Submasks including zero

JAVA

```java
int sub = mask;
while (true) {
    process(sub);
    if (sub == 0) {
        break;
    }
    sub = (sub - 1) & mask;
}
```

This handles `mask == 0` and includes zero exactly once.

C18 - Gray codes

Generate `i ^ (i >>> 1)`. For adjacent values, assert `Integer.bitCount(previous ^ current) == 1`. Limit `n` before calculating `1 << n`.

C19 - Maximum XOR differential test

For small random nonnegative arrays, compare the trie result with the maximum over all `i < j`. Include duplicates, zeros, one-bit values, and values near `Integer.MAX_VALUE`. A mismatch exposes trie or contract defects.

C20 - Range AND alternatives

Common-prefix method shifts both endpoints until equal, then shifts back. Clearing method repeatedly applies `right &= right - 1` while `left < right`. Verify both against a direct small-range baseline.

C21 - Total set bits

Use the highest-power block recurrence and a `long` result. Test against a direct `Long.bitCount` sum for many small `n` values.

C22 - Minimum XOR pair

Clone to preserve ownership, reject or define negative semantics, sort, then inspect adjacent pairs. Complexity is `O(n log n)` time and `O(n)` copy space.

C23 - Distinct subarray OR

For each ending position, build a set containing the value and every `prior | value`. Add the frontier to a global set. Explain the width-bounded frontier due to monotonic bit addition.

C24 - Versioned permissions

A strong design includes a named enum or constants, one mapping from name to stable bit, a known-mask validator, explicit schema version, serialization byte order, reserved positions, and `AtomicLong.getAndUpdate` or a lock for compound updates. Tests cover unknown bits and concurrent independent enables.

Part E - Follow-up model answers

F01. On fixed `int`, at most 32 positions are processed, so the bound is constant with respect to array length. For a `b`-bit arbitrary value, scanning costs `O(b)` and Kernighan costs `O(p)` selected positions, with `p <= b`.

F02. Reject both. Java would silently convert 64 to zero and -1 to 63 for a `long` shift, but those are not valid logical indexes.

F03. Zero makes the raw clear-lowest identity zero, and MIN_VALUE has exactly one representation bit. A positive mathematical-domain condition rejects both.

F04. Use a frequency map and verify exactly one count one and every other count two. That changes auxiliary space to `O(n)` and may be necessary at an untrusted boundary.

F05. The isolated mask is negative as an `int`, but AND partitioning still works. Treat it as a pattern and do not take its absolute value.

F06. A static prefix array no longer supports constant-time updates. Consider a Fenwick tree or segment tree with XOR as the associative operation, depending on required update/query contracts.

F07. At most `n(n + 1)/2` subarrays match, which exceeds `int` for sufficiently large `n`. Prefix state remains `int`; the count should be `long`.

F08. Materialization becomes impractical around low twenties for many object-heavy Java outputs, long before a 64-bit representation limit. The exact threshold depends on output form and resource budget.

F09. The mask omits order and any attributes not needed by future transitions. It is safe only when the future cost depends solely on the selected set and a derivable step such as `bitCount(mask)`.

F10. Object nodes are simplest but allocate heavily; primitive child arrays improve locality and memory; quadratic search uses no trie and can win for tiny input. State `O(nw)` trie cost versus `O(n^2)` baseline.

F11. AND is monotonic under expansion: bits only disappear. XOR bits can alternate, so neither a common-prefix result nor a width-bounded monotonic frontier follows.

F12. `BitSet` is mutable, and methods such as `and`, `or`, `xor`, and `andNot` modify the receiver. Clone when inputs must remain unchanged and avoid exposing mutable internal state.

F13. Assign stable meanings, reserve bits, encode a schema version, and define whether unknown bits are rejected, preserved, or cleared. Never reinterpret an old persisted bit for a new permission.

F14. Use a lock or `AtomicLong.getAndUpdate(current -> current | mask)`. `volatile` alone does not make the workflow atomic.

F15. Byte order defines how the eight bytes map to the 64-bit numeric value. Bit numbering defines meanings within that value. Document both and convert at the boundary with a specified `ByteOrder`.

Part F - Assessment review guides

Assessment 1

A passing answer shows the complete sign-extended 32-bit reasoning for `-13`, distinguishes arithmetic and logical right shift, validates both ends of the `long` index range, proves low-bit identities using subtraction and two's complement, and explains that `~byte` produces `int`.

Assessment 2

Look for an explicit occurrence promise, one loop invariant per reduction, half-open prefix endpoints, initial zero prefix in frequency counting, zero-safe submask termination, and honest expected-hash versus worst-case qualifications.

Assessment 3

A strong response starts each optimization from a baseline, defines signedness, makes ownership and mutation explicit, discusses trie allocation and primitive alternatives, separates mask representation from concurrency, and refuses infeasible subset output even when a wider primitive is available.

Final correction checklist

If any answer was wrong, classify the cause:

- representation width;
- signed versus unsigned interpretation;
- invalid shift distance;
- wrong operator truth table;
- missing occurrence contract;
- incorrect prefix endpoints;
- output-space omission;
- exponential feasibility;
- mutation or ownership;
- concurrency; or
- schema/versioning.

Repeat the smallest matching example, then solve a different problem with the same failure mode. The goal is to correct the mental model, not memorize the answer key.

SDE-2 CORE CHAPTER

Chapter 9 - Forty Executable Bit-Manipulation Checks

This dependency-free Java 21 companion turns the volume's core identities, XOR patterns, subset methods, range techniques, and packed-field contracts into forty executable checks. Compile with javac -Xlint:all -Werror BitManipulationExamples.java and run with java BitManipulationExamples. A successful run prints exactly PASS 40 Bit Manipulation checks.

JAVA (continued 1/13)

```java
import java.util.Arrays;
import java.util.BitSet;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.atomic.AtomicLong;

public final class BitManipulationExamples {
    private static int checks;

    private BitManipulationExamples() {
    }

    static String bits32(int value) {
        StringBuilder result = new StringBuilder(Integer.SIZE);
        for (int bit = Integer.SIZE - 1; bit >= 0; bit--) {
            result.append((value >>> bit) & 1);
        }
        return result.toString();
    }

    static long oneBit(int index) {
        if (index < 0 || index >= Long.SIZE) {
            throw new IllegalArgumentException("index must be in [0, 63]");
        }
        return 1L << index;
    }

    static boolean isSet(long value, int index) {
        return (value & oneBit(index)) != 0;
    }

    static long set(long value, int index) {
        return value | oneBit(index);
    }
```

JAVA (continued 2/13)

```
    static long clear(long value, int index) {
        return value & ~oneBit(index);
    }

    static long toggle(long value, int index) {
        return value ^ oneBit(index);
    }

    static long lowBitsMask(int width) {
        if (width < 0 || width > Long.SIZE) {
            throw new IllegalArgumentException("width must be in [0, 64]");
        }
        if (width == 0) {
            return 0L;
        }
        return width == Long.SIZE ? -1L : (1L << width) - 1;
    }

    static long extractField(long word, int offset, int width) {
        validateField(offset, width);
        return width == Long.SIZE
                ? word
                : (word >>> offset) & lowBitsMask(width);
    }

    static long replaceField(long word, int offset, int width, long value) {
        validateField(offset, width);
        long lowMask = lowBitsMask(width);
        if ((value & ~lowMask) != 0) {
            throw new IllegalArgumentException("value does not fit field");
        }
        if (width == Long.SIZE) {
            return value;
        }
        long mask = lowMask << offset;
```

JAVA (continued 3/13)

```
        return (word & ~mask) | (value << offset);
    }

    private static void validateField(int offset, int width) {
        if (offset < 0 || width < 1 || width > Long.SIZE
                || offset > Long.SIZE - width) {
            throw new IllegalArgumentException("invalid field bounds");
        }
    }

    static int countSetBits(int value) {
        int count = 0;
        while (value != 0) {
            value &= value - 1;
            count++;
        }
        return count;
    }

    static boolean isPowerOfTwo(int value) {
        return value > 0 && (value & (value - 1)) == 0;
    }

    static boolean isPowerOfFour(int value) {
        return isPowerOfTwo(value) && (value & 0x5555_5555) != 0;
    }

    static int hammingDistance(int first, int second) {
        return Integer.bitCount(first ^ second);
    }

    static int reverseBits(int value) {
        int reversed = 0;
        for (int bit = 0; bit < Integer.SIZE; bit++) {
            reversed = (reversed << 1) | (value & 1);
            value >>>= 1;
```

JAVA (continued 4/13)

```
        }
        return reversed;
    }

    static int[] countBitsThrough(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be nonnegative");
        }
        int[] bits = new int[n + 1];
        for (int value = 1; value <= n; value++) {
            bits[value] = bits[value & (value - 1)] + 1;
        }
        return bits;
    }

    static int singleAmongPairs(int[] values) {
        requireNonEmpty(values);
        int answer = 0;
        for (int value : values) {
            answer ^= value;
        }
        return answer;
    }

    static int missingFromZeroThroughN(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
        int answer = values.length;
        for (int index = 0; index < values.length; index++) {
            answer ^= index;
            answer ^= values[index];
        }
        return answer;
    }
```

JAVA (continued 5/13)

```
    static int[] twoSinglesAmongPairs(int[] values) {
        requireNonEmpty(values);
        int combined = 0;
        for (int value : values) {
            combined ^= value;
        }
        if (combined == 0) {
            throw new IllegalArgumentException("distinct singles required");
        }
        int distinguishing = combined & -combined;
        int first = 0;
        int second = 0;
        for (int value : values) {
            if ((value & distinguishing) == 0) {
                first ^= value;
            } else {
                second ^= value;
            }
        }
        return first <= second
                ? new int[] {first, second}
                : new int[] {second, first};
    }

    static int singleAmongTriples(int[] values) {
        requireNonEmpty(values);
        int answer = 0;
        for (int bit = 0; bit < Integer.SIZE; bit++) {
            int count = 0;
            for (int value : values) {
                count += (value >>> bit) & 1;
            }
            if (count % 3 != 0) {
                answer |= 1 << bit;
            }
        }
```

JAVA (continued 6/13)

```
        return answer;
    }

    static int[] buildPrefixXor(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
        int[] prefix = new int[values.length + 1];
        for (int index = 0; index < values.length; index++) {
            prefix[index + 1] = prefix[index] ^ values[index];
        }
        return prefix;
    }

    static int rangeXor(int[] prefix, int left, int right) {
        if (prefix == null || left < 0 || right < left
                || right + 1 >= prefix.length) {
            throw new IllegalArgumentException("invalid prefix or range");
        }
        return prefix[right + 1] ^ prefix[left];
    }

    static int xorZeroThrough(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be nonnegative");
        }
        return switch (n & 3) {
            case 0 -> n;
            case 1 -> 1;
            case 2 -> n + 1;
            default -> 0;
        };
    }

    static int xorRange(int left, int right) {
        if (left < 0 || right < left) {
```

JAVA (continued 7/13)

```
            throw new IllegalArgumentException("invalid nonnegative range");
        }
        return xorZeroThrough(right)
                ^ (left == 0 ? 0 : xorZeroThrough(left - 1));
    }

    static long countSubarraysWithXor(int[] values, int target) {
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
        Map<Integer, Integer> frequency = new HashMap<>();
        frequency.put(0, 1);
        int prefix = 0;
        long count = 0;
        for (int value : values) {
            prefix ^= value;
            count += frequency.getOrDefault(prefix ^ target, 0);
            frequency.merge(prefix, 1, Integer::sum);
        }
        return count;
    }

    static int countSubmasksIncludingZero(int mask) {
        int count = 0;
        int sub = mask;
        while (true) {
            count++;
            if (sub == 0) {
                return count;
            }
            sub = (sub - 1) & mask;
        }
    }

    static int grayCode(int index) {
```

JAVA (continued 8/13)

```
        if (index < 0) {
            throw new IllegalArgumentException("index must be nonnegative");
        }
        return index ^ (index >>> 1);
    }

    private static final class BitNode {
        private final BitNode[] next = new BitNode[2];
    }

    static int maximumXorPair(int[] values) {
        if (values == null || values.length < 2) {
            throw new IllegalArgumentException("at least two values required");
        }
        BitNode root = new BitNode();
        for (int value : values) {
            if (value < 0) {
                throw new IllegalArgumentException("nonnegative values required");
            }
            BitNode node = root;
            for (int bit = 30; bit >= 0; bit--) {
                int current = (value >>> bit) & 1;
                if (node.next[current] == null) {
                    node.next[current] = new BitNode();
                }
                node = node.next[current];
            }
        }
        int best = 0;
        for (int value : values) {
            BitNode node = root;
            int candidate = 0;
            for (int bit = 30; bit >= 0; bit--) {
                int current = (value >>> bit) & 1;
                int preferred = current ^ 1;
```

JAVA (continued 9/13)

```
                if (node.next[preferred] != null) {
                    candidate |= 1 << bit;
                    node = node.next[preferred];
                } else {
                    node = node.next[current];
                }
            }
            best = Math.max(best, candidate);
        }
        return best;
    }

    static int rangeBitwiseAnd(int left, int right) {
        if (left < 0 || right < left) {
            throw new IllegalArgumentException("invalid nonnegative range");
        }
        int shifts = 0;
        while (left != right) {
            left >>>= 1;
            right >>>= 1;
            shifts++;
        }
        return left << shifts;
    }

    static long totalSetBitsThrough(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n must be nonnegative");
        }
        if (n == 0) {
            return 0;
        }
        int highestBit = 31 - Integer.numberOfLeadingZeros(n);
        int power = 1 << highestBit;
        long fullBlock = highestBit == 0
```

JAVA (continued 10/13)

```
                ? 0
                : (long) highestBit * (power >>> 1);
        return fullBlock + (long) n - power + 1
                + totalSetBitsThrough(n - power);
    }

    static int significantComplement(int value) {
        if (value < 0) {
            throw new IllegalArgumentException("nonnegative value required");
        }
        if (value == 0) {
            return 1;
        }
        int highest = Integer.highestOneBit(value);
        int mask = (highest << 1) - 1;
        return value ^ mask;
    }

    static int minimumXorPair(int[] input) {
        if (input == null || input.length < 2) {
            throw new IllegalArgumentException("at least two values required");
        }
        int[] values = input.clone();
        for (int value : values) {
            if (value < 0) {
                throw new IllegalArgumentException("nonnegative values required");
            }
        }
        Arrays.sort(values);
        int best = Integer.MAX_VALUE;
        for (int index = 1; index < values.length; index++) {
            best = Math.min(best, values[index - 1] ^ values[index]);
        }
        return best;
    }
```

JAVA (continued 11/13)

```
    static int distinctSubarrayOrCount(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
        Set<Integer> all = new HashSet<>();
        Set<Integer> previous = Set.of();
        for (int value : values) {
            Set<Integer> current = new HashSet<>();
            current.add(value);
            for (int prior : previous) {
                current.add(prior | value);
            }
            all.addAll(current);
            previous = current;
        }
        return all.size();
    }

    static int addWithBits(int first, int second) {
        while (second != 0) {
            int carry = (first & second) << 1;
            first ^= second;
            second = carry;
        }
        return first;
    }

    private static void requireNonEmpty(int[] values) {
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException("values must not be empty");
        }
    }

    private static void check(boolean condition) {
```

JAVA (continued 12/13)

```
        if (!condition) {
            throw new AssertionError("check " + (checks + 1) + " failed");
        }
        checks++;
    }

    private static void checkThrows(Runnable action) {
        try {
            action.run();
        } catch (IllegalArgumentException expected) {
            checks++;
            return;
        }
        throw new AssertionError("check " + (checks + 1) + " failed");
    }

    public static void main(String[] args) {
        check(bits32(5).equals("00000000000000000000000000000101"));
        check(bits32(-1).equals("11111111111111111111111111111111"));
        check(isSet(1L << 63, 63));
        check(set(0, 40) == 1L << 40);
        check(clear(0b1111, 2) == 0b1011);
        check(toggle(toggle(123, 6), 6) == 123);
        checkThrows(() -> oneBit(64));
        check(lowBitsMask(0) == 0);
        check(lowBitsMask(5) == 0b1_1111);
        check(lowBitsMask(64) == -1L);
        check(extractField(0b1101_0110, 2, 3) == 5);
        check(replaceField(0b1111_0000, 4, 4, 5) == 0b0101_0000);
        checkThrows(() -> replaceField(0, 3, 2, 4));
        check(countSetBits(0) == 0);
        check(countSetBits(-1) == 32);
        check(isPowerOfTwo(1 << 30));
        check(!isPowerOfTwo(0));
        check(!isPowerOfTwo(Integer.MIN_VALUE));
```

JAVA (continued 13/13)

```
        check(isPowerOfFour(64));
        check(!isPowerOfFour(8));
        check(hammingDistance(0b1010, 0b0011) == 2);
        check(reverseBits(1) == Integer.MIN_VALUE);
        check(Arrays.equals(countBitsThrough(5), new int[] {0, 1, 1, 2, 1, 2}));
        check(singleAmongPairs(new int[] {7, -2, 7}) == -2);
        check(missingFromZeroThroughN(new int[] {3, 0, 1}) == 2);
        check(Arrays.equals(
                twoSinglesAmongPairs(new int[] {1, 2, 1, 3, 2, 5}),
                new int[] {3, 5}));
        check(singleAmongTriples(new int[] {6, -9, 6, 6}) == -9);
        int[] prefix = buildPrefixXor(new int[] {4, 2, 7, 2});
        check(rangeXor(prefix, 1, 3) == 7);
        check(xorZeroThrough(6) == 7);
        check(xorRange(3, 6) == (3 ^ 4 ^ 5 ^ 6));
        check(countSubarraysWithXor(new int[] {4, 2, 2, 6, 4}, 6) == 4);
        check(countSubmasksIncludingZero(0b10110) == 8);
        check(Integer.bitCount(grayCode(6) ^ grayCode(7)) == 1);
        check(maximumXorPair(new int[] {3, 10, 5, 25, 2, 8}) == 28);
        check(rangeBitwiseAnd(26, 30) == 24);
        check(totalSetBitsThrough(13) == 25);
        check(significantComplement(5) == 2);
        check(minimumXorPair(new int[] {9, 5, 3}) == 6);
        check(distinctSubarrayOrCount(new int[] {1, 2}) == 3);
        AtomicLong flags = new AtomicLong();
        flags.getAndUpdate(current -> current | (1L << 7));
        BitSet bitSet = BitSet.valueOf(new long[] {flags.get()});
        check(bitSet.get(7) && addWithBits(-4, 9) == 5);

        if (checks != 40) {
            throw new AssertionError("expected 40 checks, found " + checks);
        }
        System.out.println("PASS 40 Bit Manipulation checks");
    }
}
```

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: draw bit patterns and trace operators before coding
- Interview Core: implement masks, bit counts, power tests, XOR families, and range queries
- SDE-2 Follow-up: prove invariants, handle signed boundaries, and compare alternative representations
- Challenge: solve submask, bitmask-state, maximum-XOR, and packed-field problems under time

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous volume: 01B - Number Systems Interview Patterns and Rapid Revision

Series index

Next volume: 05 - Loop Mastery, Patterns, and Index Calculations

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Learning Step 4 - Volume 4 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.